المملكة المغربية
وزارة التعليم العالي
و البحث العلمي و الإبتكار

جامعة ابن طفيل
Ibn Tofaïl University

Mémoire de projet de fin d'études

Pour l'obtention du diplôme

Master : Réseaux et Services Mobiles (RSM)

Suivi des Dossiers Médicaux Décentralisés Avec Détection de Situations Critiques

Réalisé par :

SOUAD TAHRI

Soutenu le : 12 Janvier 2026

Devant le jury composé de :

Pr. Adil ELMAKRANI	Encadrant	PES à FS Kénitra.
Pr. LAASSIRI Jalal	Président	PES à FS Kénitra.
Pr. DERROUZ Hatim	Examinateur	MC à FS Kénitra.

Promotion : 2023- 2025

Dédicaces

Je dédie ce modeste travail à toutes les personnes qui ont contribué, de près ou de loin, à l'aboutissement de ce projet.

À ma famille, pour leur amour inconditionnel, leur soutien moral et leur confiance tout au long de mon parcours.

À mes amis sincères, pour leur présence, leurs encouragements et leurs mots de motivation dans les moments de doute.

À toute personne qui a cru en moi et m'a poussé à donner le meilleur de moi-même.

Merci à vous.

Remerciements

Je remercie en premier lieu Allah pour m'avoir permis d'exercer ma force et ma patience dans l'achèvement de ce projet de fin d'études.

Je tiens à exprimer ma profonde gratitude à Monsieur le Professeur Adil ELMAKRANI, mon encadrant, pour son accompagnement, ses conseils, et son soutien durant tout ce travail.

Je remercie Monsieur le Professeur LAASSIRI Jalal, Président du jury ainsi que les membres jury examinateurs, de l'honneur qu'ils m'ont fait de bien vouloir évaluer ce projet.

Et pour terminer, je remercie ma famille et toute personne ayant contribué, de près comme de loin à l'aboutissement de ce travail.

Résumé

Ce projet porte sur la conception et le développement d'une plateforme hybride de gestion et de suivi des dossiers médicaux, combinant la blockchain et l'intelligence artificielle afin de renforcer la confidentialité, l'intégrité et la traçabilité des informations cliniques. Dans un contexte d'e-santé où les données sont sensibles et souvent centralisées, la solution proposée vise à limiter les risques de falsification, de perte et d'accès non autorisé grâce à une gouvernance décentralisée. L'architecture repose sur une blockchain Ethereum utilisant des smart contracts en Solidity pour orchestrer le contrôle d'accès, où un administrateur central gère les droits des médecins tandis que les praticiens autorisés peuvent enregistrer les diagnostics. Le stockage des données volumineuses est assuré par le système IPFS, ne conservant sur la chaîne que le CID unique qui garantit l'intégrité absolue des fichiers. Parallèlement, un module d'intelligence artificielle basé sur l'algorithme Decision Tree et intégré via Flask permet d'analyser les paramètres cliniques pour assister les médecins dans la détection de cas critiques. La plateforme offre des interfaces adaptées à chaque acteur, permettant à l'administrateur de réguler le réseau, au médecin de générer des diagnostics sécurisés et au patient de consulter son historique médical en toute confidentialité. Les expérimentations menées sur un jeu de données de pathologie cardiaque confirment l'efficacité de cette approche intégrée pour sécuriser et automatiser le suivi médical moderne.

Mots-clés : Blockchain, Ethereum, Smart contract, IPFS, Decision Tree, Dossier médical électronique, Contrôle d'accès, Traçabilité, Intelligence artificielle, Machine learning, Confidentialité.

Abstract

This project focuses on the design and development of a hybrid platform for the management and monitoring of medical records, combining blockchain technology and artificial intelligence to enhance the confidentiality, integrity, and traceability of clinical information. In an e-health context where data is sensitive and often centralized, the proposed solution aims to mitigate the risks of falsification, loss, and unauthorized access through decentralized governance. The architecture is based on an Ethereum blockchain utilizing smart contracts written in Solidity to orchestrate access control, where a central administrator manages the rights of doctors, while authorized practitioners can record diagnostics. Voluminous data storage is handled by the IPFS system, storing only the unique CID on-chain to guarantee the absolute integrity of the files. Simultaneously, an artificial intelligence module based on the Decision Tree algorithm and integrated via Flask enables the analysis of clinical parameters to assist physicians in detecting critical cases. The platform provides tailored interfaces for each actor, allowing the administrator to regulate the network, the doctor to generate secure diagnostics, and the patient to consult their medical history in total confidentiality. Experiments conducted on a cardiac pathology dataset confirm the effectiveness of this integrated approach in securing and automating modern medical follow-ups.

Keywords : Blockchain, Ethereum, Smart contract, IPFS, Decision Tree, Electronic Health Record, Access Control, Traceability, Artificial Intelligence, Machine Learning, Confidentiality.

Liste des acronymes

Table 1. Liste des acronymes

Acronyme	Signification
AI	Artificial Intelligence
API	Application Programming Interface
CID	Content Identifier (Identifiant de contenu unique sur IPFS)
CSV	Comma-Separated Values (Format de fichier pour les bases de données)
CSS	Cascading Style Sheets
DApp	Decentralized Application (Application décentralisée)
DME	Dossier Médical Électronique
EHR	Electronic Health Record
EVM	Ethereum Virtual Machine
GAS	Unité de mesure de l'effort computationnel sur Ethereum
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
IPFS	InterPlanetary File System
JSON	JavaScript Object Notation
KNN	K-Nearest Neighbors
ML	Machine Learning (Apprentissage Automatique)
PES	Professeur de l'Enseignement Supérieur
REST	Representational State Transfer
RSM	Réseaux et Services Mobiles
SVM	Support Vector Machine
UI/UX	User Interface / User Experience
UML	Unified Modeling Language
URI	Uniform Resource Identifier
VS Code	Visual Studio Code
Web3	Le Web décentralisé basé sur la Blockchain

Table des matières

Dédicaces ii

Remerciements ii

Résumé iv

Abstract v

Liste des acronymes vi

Introduction générale 1

1 Étude Générale et Conceptuelle 3

1.1 Présentation du domaine médical 3
1.2 Problématique 4
1.3 Solution proposée 5
1.4 Objectifs du projet 6
1.5 Méthodologie de travail 7
1.6 Conclusion 8

2 Étude des Technologies Utilisées 9

2.1 Blockchain 9
2.2 Smart Contracts avec Solidity 11
2.3 Truffle & Ganache 12
2.4 IPFS (InterPlanetary File System) 14
2.5 Intelligence Artificielle 14
2.6 Flask 16
2.7 Web Technologies 16
2.8 Conclusion 18

3 Analyse et Conception du Système **19**
3.1 Analyse des besoins 19
3.1.1 Acteurs du système 19
3.1.2 Besoins fonctionnels 20
3.1.3 Besoins non fonctionnels 21
3.2 Architecture globale du système 22
3.3 Modélisation UML 24
3.3.1 Diagramme de cas d'utilisation 24
3.3.2 Diagrammes de séquence 25
3.4 Conclusion 28

4 Implémentation de la Solution **29**
4.1 Implémentation de la Blockchain 29
4.2 Déploiement avec Truffle 31
4.3 Implémentation du module d'Intelligence Artificielle 32
4.4 API Flask 34
4.5 Développement de l'Interface Utilisateur (Frontend) 35
4.6 Conclusion 37

5 Tests, Résultats et Discussion **38**
5.1 Tests du Smart Contract 38
5.2 Tests du modèle IA 39
5.3 Tests du système global 42
5.3.1 Communication entre les modules 42
5.3.2 Scénarios réels 43
5.4 Analyse et discussion des résultats 48
5.5 Perspectives futures 48
5.6 Conclusion 49

6 Conclusion Générale **53**

A Le Smart Contract (Solidity) **54**
A.1 Structures de données et stockage 54
A.2 Contrôle d'accès : Modifiers et logique de sécurité 55
A.3 Fonctions principales (liées aux besoins fonctionnels) 55

B Implémentation de l'Intelligence Artificielle **59**

B.1 Entraînement du modèle 59
B.2 API de prédiction 60
B.3 Évaluation et Visualisation 60

C Structure du fichier JSON médical stocké sur IPFS **62**

D Visualisation des Transactions sur Ganache **63**

Table des figures

3.1 architecture globale du système. 22
3.2 Diagramme global de cas d'utilisation. 25
3.3 Diagramme de séquence : mise à jour du dossier (médecin). 26
3.4 Diagramme de séquence : consultation du dossier par le patient. 27
3.5 autorisation d'un médecin par l'administrateur. 27

5.1 Comparaison des performances des modèles de classification. 40
5.2 Matrice de confusion du modèle Decision Tree. 41
5.3 Importance des variables cliniques dans le processus de décision. 42
5.4 Interface d'administration et autorisation d'un professionnel de santé. 43
5.5 Notification de succès et mise à jour de la liste des médecins. 44
5.6 Interface de saisie des données cliniques pour l'analyse prédictive. 45
5.7 Notification de détection de risque critique par le modèle IA. 45
5.8 Confirmation du téléversement sur IPFS et génération du CID. 46
5.9 Tableau de bord principal de l'interface Patient. 47
5.10 Vue détaillée du profil personnel du patient. 47
5.11 Rapport médical détaillé récupéré depuis IPFS 48

A.1 Structure Doctor (extrait du Smart Contract). 54
A.2 Structure Patient (extrait du Smart Contract). 54
A.3 Mécanismes de stockage et d'accès (arrays + mappings). 55
A.4 Fonction addDoctor() (extrait du Smart Contract). 56
A.5 Fonction authorizeDoctor() (extrait du Smart Contract). 56
A.6 Fonction revokeDoctor() (extrait du Smart Contract). 56
A.7 Fonction addPatient() (extrait du Smart Contract). 57
A.8 Fonction updatePatientDataHash() (extrait du Smart Contract). 57
A.9 Fonction getPatientRecord() (extrait du Smart Contract). 58
A.10 Fonction getMyPatientsAddresses() (extrait du Smart Contract). 58

B.1 Comparaison multi-algorithmes et sélection du modèle de classification optimal. 59
B.2 Fonction getMyPatientsAddresses() (extrait du Smart Contract). . . 60
B.3 Génération de la matrice de confusion pour l'évaluation des performances diagnostiques. 60

C.1 Structure des métadonnées médicales au format JSON (Stockage IPFS). 62

D.1 Processus de déploiement du Smart Contract MedicalRecords.sol sur Ganache via Truffle. 63

Liste des tableaux

1 Liste des acronymes vi

6.1 Comparaison entre les systèmes classiques et notre solution proposée . 51

Introduction générale

Avec l'évolution rapide des technologies de l'information et de la communication, le secteur médical connaît une transformation majeure marquée par la digitalisation des services de santé. Les dossiers médicaux, autrefois conservés sous forme papier, sont aujourd'hui de plus en plus numérisés afin de faciliter l'accès à l'information, d'améliorer la qualité des soins et d'optimiser la gestion des données des patients. Cette transformation permet aux professionnels de santé de consulter rapidement l'historique médical des patients, de réduire les erreurs médicales et d'assurer un meilleur suivi des traitements. Toutefois, malgré ces avancées technologiques, les systèmes actuels présentent encore plusieurs limites, notamment en matière de sécurité, de confidentialité et de gestion des données médicales sensibles.

En effet, la majorité des systèmes de gestion des dossiers médicaux existants reposent sur une architecture centralisée, dans laquelle les données sont stockées sur des serveurs contrôlés par une seule entité. Cette centralisation engendre plusieurs problématiques importantes, telles que la vulnérabilité aux cyberattaques, aux fuites de données et aux pannes techniques. Une seule faille de sécurité peut ainsi compromettre un grand nombre de dossiers médicaux et mettre en danger la confidentialité des patients. De plus, le patient dispose souvent d'un contrôle limité sur ses propres données, ce qui soulève des questions éthiques et juridiques liées à la protection de la vie privée. Par ailleurs, ces systèmes ne garantissent pas toujours une traçabilité fiable des accès et des modifications effectuées sur les dossiers médicaux, et l'analyse intelligente des données pour détecter automatiquement des situations critiques reste généralement absente ou insuffisante.

Face à ces limites, l'utilisation de la Blockchain apparaît comme une solution innovante pour renforcer la sécurité et la transparence dans la gestion des dossiers médicaux. Grâce à son architecture décentralisée, la Blockchain permet d'assurer l'intégrité, l'immuabilité et la traçabilité des données médicales, tout en réduisant les risques liés à la centralisation. En parallèle, l'intégration de l'Intelligence Artificielle, notamment à travers des techniques de Machine Learning, permet d'analyser les données médicales afin de détecter automatiquement des situations critiques chez les patients. Cette combinaison offre un système à la fois sécurisé et intelligent, capable d'assister les professionnels de santé dans la prise de décision. Dans ce contexte, l'objectif principal de ce projet est de concevoir et de développer une plateforme décentralisée de suivi des dossiers médicaux basée sur la Blockchain et l'Intelligence Artificielle, visant à assurer la sécurité et la confidentialité des données, garantir leur traçabilité, faciliter la gestion décentralisée des dossiers médicaux et intégrer un module intelligent de détection des situations médicales critiques, tout en proposant une interface web simple et intuitive destinée aux médecins et aux patients. Ce rapport est structuré en plusieurs chapitres. Le premier chapitre présente le contexte général du projet ainsi que la problématique abordée. Le deuxième chapitre est consacré à l'analyse des besoins et à la spécification des exigences fonctionnelles et techniques. Le troisième chapitre

décrit la conception globale de la solution, incluant l'architecture du système et les choix technologiques. Le quatrième chapitre détaille l'implémentation de la plateforme et des différents modules développés. Le cinquième chapitre regroupe les tests réalisés, les résultats obtenus ainsi que leur analyse et discussion, et présente également les perspectives d'amélioration du système.

Chapitre 1

Étude Générale et Conceptuelle

Ce chapitre a pour objectif de présenter le cadre général du projet ainsi que les concepts fondamentaux liés à la gestion des dossiers médicaux. Il met en évidence les principales problématiques des systèmes actuels et introduit l'apport des technologies émergentes, notamment la Blockchain et l'Intelligence Artificielle, comme solutions potentielles pour améliorer la sécurité, la traçabilité et l'efficacité du suivi médical.

1.1 Présentation du domaine médical

Le domaine médical constitue l'un des secteurs les plus sensibles et les plus critiques, en raison de sa relation directe avec la santé et la vie humaine. Il repose sur la collecte, le traitement et l'exploitation de nombreuses informations médicales concernant les patients, telles que les antécédents médicaux, les diagnostics, les traitements, les analyses et les suivis cliniques. Ces informations sont regroupées au sein des dossiers médicaux, qui représentent un élément central dans le processus de prise en charge des patients.

Importance des dossiers médicaux

Les dossiers médicaux jouent un rôle fondamental dans le système de santé, car ils permettent aux professionnels de santé d'avoir une vision globale et précise de l'état de santé d'un patient. Grâce à ces dossiers, les médecins peuvent consulter l'historique médical, suivre l'évolution des maladies, adapter les traitements et prendre des décisions médicales plus appropriées. Les dossiers médicaux contribuent également à améliorer la coordination entre les différents acteurs du système de santé, notamment en facilitant le partage d'informations entre médecins, spécialistes et établissements hospitaliers.

Par ailleurs, les dossiers médicaux constituent une source précieuse de données pour la recherche médicale et l'analyse statistique. L'exploitation de ces données permet d'améliorer la qualité des soins, de mieux comprendre certaines pathologies et de développer de nouvelles approches thérapeutiques. Ainsi, une gestion efficace et fiable des dossiers médicaux est indispensable pour garantir un système de santé performant et de qualité.

Problèmes actuels : sécurité, confidentialité et perte de données

Malgré leur importance, les dossiers médicaux font face à plusieurs problématiques majeures dans les systèmes actuels. L'un des principaux défis concerne la sécurité des données médicales. Ces données étant extrêmement sensibles, elles constituent une cible privilégiée pour les cyberattaques et les accès non autorisés. Les systèmes traditionnels ne garantissent pas toujours un niveau de protection suffisant contre les intrusions, ce qui peut entraîner des violations de données graves.

La confidentialité représente également un enjeu crucial dans le domaine médical. Les patients ont le droit de contrôler l'accès à leurs informations personnelles, cependant, dans de nombreux systèmes existants, ce contrôle reste limité. Les accès aux dossiers médicaux ne sont pas toujours tracés de manière transparente, ce qui rend difficile la détection des abus ou des utilisations non autorisées des données.

En outre, la perte de données médicales constitue un autre problème important. Les systèmes centralisés sont vulnérables aux pannes techniques, aux erreurs humaines et aux catastrophes matérielles, pouvant entraîner la perte totale ou partielle des dossiers médicaux. Une telle perte peut avoir des conséquences graves sur la continuité des soins et la sécurité des patients. Ces différentes problématiques montrent la nécessité de repenser les méthodes traditionnelles de gestion des dossiers médicaux et d'adopter des solutions plus sécurisées et fiables.

1.2 Problématique

Malgré les avancées significatives réalisées dans le domaine de la digitalisation du secteur médical, la gestion des dossiers médicaux demeure confrontée à plusieurs problématiques majeures qui limitent l'efficacité et la fiabilité des systèmes existants. Ces problématiques concernent principalement la centralisation des données médicales, le manque de transparence et de traçabilité, ainsi que la difficulté de détecter précocement les situations médicales critiques.

Centralisation des données médicales

La plupart des systèmes actuels de gestion des dossiers médicaux reposent sur une architecture centralisée, dans laquelle l'ensemble des données est stocké sur des serveurs contrôlés par une seule entité, telle qu'un hôpital ou un organisme de santé. Cette centralisation constitue un point de faiblesse important, car elle crée une dépendance totale à une infrastructure unique. En cas de panne, de cyberattaque ou de défaillance technique, l'accès aux dossiers médicaux peut être compromis, voire totalement interrompu.

De plus, la centralisation limite la disponibilité et la flexibilité des données médicales, notamment lorsqu'un patient change d'établissement de santé ou consulte plusieurs professionnels médicaux. Cela peut entraîner des retards dans la prise en charge du patient et une fragmentation de son historique médical, affectant ainsi la qualité des soins.

Manque de transparence et de traçabilité

Un autre problème majeur des systèmes centralisés réside dans le manque de transparence et de traçabilité des accès aux dossiers médicaux. Dans de nombreux cas, il est difficile de savoir précisément qui a consulté ou modifié un dossier médical, à quel moment et pour quelle raison. Cette absence de traçabilité claire rend complexe la détection des accès non autorisés ou des manipulations abusives des données.

Par ailleurs, les patients disposent souvent d'un contrôle limité sur leurs propres données médicales. Ils ne peuvent pas toujours vérifier l'utilisation de leurs informations personnelles ni gérer les autorisations d'accès accordées aux professionnels de santé. Cette situation soulève des préoccupations importantes en matière de respect de la vie privée, de conformité réglementaire et de confiance dans les systèmes de santé numériques.

Difficulté de détection précoce des cas critiques

En plus des problématiques liées à la gestion et à la sécurité des données, les systèmes traditionnels présentent également des limites en matière d'analyse intelligente des dossiers médicaux. La détection précoce des situations médicales critiques repose généralement sur l'intervention humaine, ce qui peut entraîner des retards ou des erreurs, notamment en cas de surcharge de travail des professionnels de santé.

L'absence de mécanismes automatisés d'analyse et de prédiction empêche l'exploitation efficace des données médicales disponibles. Ainsi, certaines situations critiques pouvant être anticipées à partir des antécédents médicaux ou des paramètres cliniques ne sont pas détectées à temps, ce qui peut avoir des conséquences graves sur la santé des patients. Cette problématique met en évidence la nécessité d'intégrer des outils intelligents capables d'analyser les données médicales et d'assister les médecins dans la prise de décision.

1.3 Solution proposée

Afin de répondre aux problématiques identifiées dans la gestion des dossiers médicaux, notamment la centralisation des données, le manque de transparence et la difficulté de détection précoce des situations critiques, ce projet propose une solution innovante basée sur l'utilisation conjointe de la Blockchain et de l'Intelligence Artificielle, au sein d'une architecture décentralisée.

Utilisation de la Blockchain pour la sécurité

La Blockchain constitue un élément clé de la solution proposée, en raison de ses caractéristiques intrinsèques telles que la décentralisation, l'immuabilité et la transparence. En utilisant la Blockchain pour la gestion des dossiers médicaux, il devient possible de garantir l'intégrité des données, car toute modification ou ajout d'information est enregistré de manière permanente et vérifiable. Cela permet d'assurer une traçabilité

complète des opérations effectuées sur les dossiers médicaux, tout en réduisant les risques de falsification ou de manipulation des données.

De plus, la Blockchain permet de renforcer la sécurité et la confidentialité des informations médicales en limitant les accès non autorisés. Les droits d'accès peuvent être gérés de manière précise à travers des smart contracts, offrant ainsi aux patients un meilleur contrôle sur leurs données personnelles. Cette approche contribue à instaurer un climat de confiance entre les différents acteurs du système de santé.

Intégration de l'Intelligence Artificielle

En complément de la Blockchain, l'intégration de l'Intelligence Artificielle permet d'exploiter efficacement les données médicales afin d'améliorer la prise en charge des patients. Grâce aux techniques de Machine Learning, il est possible d'analyser les dossiers médicaux et d'identifier automatiquement des situations médicales critiques susceptibles de nécessiter une intervention rapide.

L'Intelligence Artificielle agit ainsi comme un outil d'aide à la décision pour les professionnels de santé, en fournissant des prédictions ou des alertes basées sur l'analyse des données disponibles. Cette automatisation permet de réduire la dépendance à l'intervention humaine seule, d'améliorer la réactivité face aux cas critiques et de diminuer le risque d'erreurs médicales.

Architecture décentralisée

La solution proposée repose sur une architecture décentralisée combinant une interface web, une Blockchain et un serveur dédié à l'Intelligence Artificielle. Cette architecture permet de séparer les responsabilités tout en assurant une communication fluide entre les différents composants du système. Le frontend permet aux utilisateurs, notamment les médecins et les patients, d'interagir avec la plateforme de manière simple et intuitive. La Blockchain assure le stockage sécurisé et la gestion des dossiers médicaux, tandis que le module d'Intelligence Artificielle se charge de l'analyse et de la détection des situations critiques.

Cette approche décentralisée améliore la robustesse, la sécurité et la scalabilité du système, tout en répondant aux exigences modernes du secteur médical en matière de protection des données et d'efficacité des soins.

1.4 Objectifs du projet

L'objectif principal de ce projet est de concevoir et de développer une plateforme intelligente et décentralisée permettant le suivi sécurisé des dossiers médicaux, tout en intégrant des mécanismes avancés d'analyse et d'aide à la décision. Cette plateforme vise à répondre aux limites des systèmes traditionnels de gestion des données médicales en exploitant les avantages de la Blockchain et de l'Intelligence Artificielle.

Plus précisément, ce projet poursuit plusieurs objectifs spécifiques. Tout d'abord, il

vise à assurer la sécurisation des dossiers médicaux en garantissant la confidentialité, l'intégrité et la protection des données sensibles des patients contre les accès non autorisés et les tentatives de falsification. L'utilisation de la Blockchain permet de renforcer la fiabilité du stockage des informations médicales et d'assurer une traçabilité complète des opérations effectuées sur les données.

Ensuite, le projet a pour objectif de mettre en place une gestion décentralisée des dossiers médicaux, afin de réduire la dépendance à une infrastructure centralisée unique et d'améliorer la disponibilité et la résilience du système. Cette approche permet également d'offrir aux patients un meilleur contrôle sur leurs données médicales, tout en facilitant le partage sécurisé des informations entre les professionnels de santé autorisés.

Un autre objectif essentiel du projet est la détection automatique des situations médicales critiques. Grâce à l'intégration de techniques de Machine Learning, le système est capable d'analyser les données médicales et d'identifier des cas à risque pouvant nécessiter une intervention rapide. Cette fonctionnalité contribue à améliorer la réactivité du personnel médical et à réduire les risques liés aux retards de diagnostic.

Enfin, le projet vise à fournir une aide à la décision médicale en mettant à disposition des professionnels de santé des outils intelligents capables de soutenir leur analyse et leur prise de décision. En combinant la sécurité de la Blockchain et les capacités analytiques de l'Intelligence Artificielle, la plateforme proposée cherche à améliorer la qualité des soins et à renforcer l'efficacité du suivi médical.

1.5 Méthodologie de travail

La réalisation de ce projet a suivi une méthodologie de travail structurée et progressive, permettant d'assurer une bonne compréhension de la problématique, un choix pertinent des technologies et une implémentation cohérente de la solution proposée. Cette méthodologie repose sur l'utilisation d'outils adaptés, des étapes de développement bien définies et des choix technologiques justifiés en fonction des objectifs du projet.

Outils utilisés

Pour la mise en œuvre de la solution, plusieurs outils et technologies ont été utilisés. La Blockchain Ethereum a été exploitée pour assurer la gestion sécurisée et décentralisée des dossiers médicaux, avec l'utilisation du langage Solidity pour le développement des smart contracts. Le déploiement et les tests des contrats intelligents ont été réalisés à l'aide des outils Truffle et Ganache, permettant un environnement de développement local fiable.

Le module d'Intelligence Artificielle a été développé en Python, en s'appuyant sur des bibliothèques de Machine Learning telles que scikit-learn, afin d'analyser les données médicales et de détecter automatiquement les situations critiques. Le framework Flask a été utilisé pour créer une API assurant la communication entre le module d'IA et l'interface web.

Enfin, le développement de l'interface utilisateur a été réalisé à l'aide des technologies web standards, notamment HTML, CSS et JavaScript, avec l'intégration de la

bibliothèque Web3.js pour permettre l'interaction avec la Blockchain via le navigateur.

Étapes de développement

Le développement du projet s'est déroulé en plusieurs étapes successives. La première étape a consisté à analyser le domaine médical et à identifier les principales problématiques liées à la gestion des dossiers médicaux. Cette phase a permis de définir clairement les besoins fonctionnels et non fonctionnels du système.

La deuxième étape a été consacrée à la conception de l'architecture globale de la solution, en définissant les rôles des différents composants, à savoir l'interface web, la Blockchain et le module d'Intelligence Artificielle. Par la suite, les smart contracts ont été développés et déployés afin de gérer les dossiers médicaux de manière sécurisée et décentralisée.

Une autre étape importante a concerné le développement et l'entraînement du modèle d'Intelligence Artificielle, ainsi que son intégration via une API Flask. Enfin, l'interface web a été développée et connectée aux différents modules du système, suivie de phases de tests et de validation afin de vérifier le bon fonctionnement global de la plateforme.

Choix technologiques

Les choix technologiques effectués dans ce projet ont été guidés par des critères de sécurité, de performance, de flexibilité et de pertinence par rapport au domaine médical. La Blockchain Ethereum a été choisie pour ses capacités à garantir l'intégrité, la traçabilité et la transparence des données. Solidity s'est imposé comme le langage adapté pour le développement des smart contracts sur cette plateforme.

Le recours à l'Intelligence Artificielle, à travers le Machine Learning, permet d'exploiter efficacement les données médicales afin d'améliorer la détection des situations critiques et d'assister les professionnels de santé dans la prise de décision. Python et Flask ont été sélectionnés pour leur simplicité, leur richesse en bibliothèques et leur large adoption dans le domaine de l'IA.

Enfin, les technologies web utilisées offrent une interface accessible, intuitive et compatible avec les navigateurs modernes, facilitant ainsi l'utilisation de la plateforme par les médecins et les patients.

1.6 Conclusion

Ce chapitre a permis de présenter le contexte général du projet et de mettre en évidence les limites des systèmes centralisés de gestion des dossiers médicaux. Il a également montré l'intérêt de l'intégration de la Blockchain et de l'Intelligence Artificielle pour assurer la sécurité, la traçabilité et l'amélioration du suivi médical. Ces éléments constituent la base conceptuelle nécessaire pour aborder, dans le chapitre suivant, la conception et l'architecture de la solution proposée.

Chapitre 2

Étude des Technologies Utilisées

Ce chapitre est consacré à la présentation des principales technologies utilisées dans la réalisation du projet. Il vise à expliquer les outils et plateformes adoptés, notamment la Blockchain, les smart contracts, l'Intelligence Artificielle et les technologies web, ainsi que leur rôle dans la mise en place d'un système sécurisé, décentralisé et intelligent de gestion des dossiers médicaux.

2.1 Blockchain

La Blockchain est une technologie émergente qui a profondément transformé la manière de stocker, de partager et de sécuriser les données numériques. Initialement introduite dans le domaine des cryptomonnaies, elle est aujourd'hui utilisée dans plusieurs secteurs, notamment la santé, en raison de ses caractéristiques avancées en matière de sécurité, de transparence et de décentralisation.

Définition et principes

La Blockchain peut être définie comme un registre distribué et décentralisé permettant d'enregistrer des transactions de manière sécurisée et immuable. Elle est constituée d'une chaîne de blocs, où chaque bloc contient un ensemble de transactions ainsi qu'un lien cryptographique vers le bloc précédent. Cette structure garantit l'intégrité des données stockées, car toute tentative de modification d'un bloc nécessiterait la modification de l'ensemble de la chaîne.

Les principaux principes de la Blockchain reposent sur la décentralisation, la transparence, l'immutabilité et la sécurité. La décentralisation signifie que les données ne sont pas stockées sur un serveur unique, mais partagées entre plusieurs nœuds du réseau. La transparence permet à tous les participants autorisés de vérifier les transactions, tandis que l'immutabilité garantit que les données enregistrées ne peuvent pas être altérées. Enfin, la sécurité est assurée par des mécanismes cryptographiques avancés.

Ethereum

Ethereum est une plateforme Blockchain open source qui va au-delà du simple transfert de cryptomonnaies. Contrairement à Bitcoin, Ethereum permet le développement et l'exécution de programmes appelés smart contracts directement sur la Blockchain. Ces programmes sont exécutés de manière autonome et décentralisée, sans intervention d'un tiers de confiance.

Ethereum repose sur une machine virtuelle appelée Ethereum Virtual Machine (EVM), qui permet l'exécution des smart contracts de façon uniforme sur l'ensemble des nœuds du réseau. Grâce à cette architecture, Ethereum est devenu l'une des plateformes les plus utilisées pour le développement d'applications décentralisées, notamment dans le domaine de la santé, où la sécurité et la traçabilité des données sont primordiales.

Smart Contracts

Les smart contracts sont des programmes informatiques stockés et exécutés sur la Blockchain. Ils permettent d'automatiser l'exécution de règles et de conditions prédéfinies sans nécessiter l'intervention d'un intermédiaire. Une fois déployés sur la Blockchain, les smart contracts deviennent immuables, ce qui garantit la fiabilité et la transparence des opérations qu'ils réalisent.

Dans le contexte de la gestion des dossiers médicaux, les smart contracts peuvent être utilisés pour contrôler l'accès aux données, gérer les autorisations des utilisateurs et assurer l'enregistrement sécurisé des informations médicales. Ils jouent ainsi un rôle essentiel dans l'automatisation et la sécurisation des processus médicaux.

Avantages de la Blockchain en santé

L'utilisation de la Blockchain dans le domaine de la santé présente plusieurs avantages significatifs. Tout d'abord, elle permet de renforcer la sécurité et la confidentialité des données médicales grâce à des mécanismes cryptographiques avancés et à une gestion décentralisée des informations. La Blockchain garantit également l'intégrité et la traçabilité des données, ce qui permet de suivre toutes les modifications effectuées sur un dossier médical.

En outre, la Blockchain favorise l'interopérabilité entre les différents acteurs du système de santé en facilitant le partage sécurisé des données entre les établissements et les professionnels autorisés. Elle offre également aux patients un meilleur contrôle sur leurs données médicales, en leur permettant de gérer les autorisations d'accès. Ces avantages font de la Blockchain une technologie particulièrement adaptée à la modernisation des systèmes de gestion des dossiers médicaux.

Mécanismes cryptographiques utilisés

Le système proposé repose sur les mécanismes cryptographiques intégrés à la blockchain Ethereum. Il utilise principalement la cryptographie asymétrique à base de courbes elliptiques, notamment l'algorithme ECDSA (Elliptic Curve Digital Signature Algorithm), pour assurer l'authentification des utilisateurs et la signature sécurisée des transactions.

Par ailleurs, des fonctions de hachage cryptographiques telles que Keccak-256 (SHA-3) sont utilisées pour garantir l'intégrité, l'immutabilité et la traçabilité des données stockées sur la blockchain ainsi que des fichiers médicaux référencés sur IPFS.

Ces mécanismes permettent d'assurer un haut niveau de sécurité sans implémentation manuelle des algorithmes de chiffrement, en s'appuyant sur l'infrastructure éprouvée d'Ethereum.

2.2 Smart Contracts avec Solidity

Les smart contracts constituent un élément fondamental des applications basées sur la Blockchain, car ils permettent d'automatiser et de sécuriser l'exécution des processus sans recourir à un intermédiaire. Dans ce projet, les smart contracts jouent un rôle central dans la gestion des dossiers médicaux décentralisés, en assurant le contrôle des accès, l'enregistrement des données et la traçabilité des opérations.

Présentation de Solidity

Solidity est un langage de programmation orienté objet spécialement conçu pour le développement de smart contracts sur la plateforme Ethereum. Il s'inspire de langages tels que JavaScript, Python et C++, ce qui le rend relativement accessible aux développeurs ayant une expérience en programmation. Solidity permet de définir des structures de données, des fonctions et des règles de gestion qui seront exécutées automatiquement sur la Blockchain.

Les smart contracts développés en Solidity sont compilés en bytecode puis déployés sur la Blockchain Ethereum, où ils sont exécutés par l'Ethereum Virtual Machine (EVM). Une fois déployés, ces contrats deviennent immuables, ce qui garantit que les règles définies ne peuvent pas être modifiées de manière arbitraire. Cette caractéristique renforce la fiabilité et la sécurité des applications décentralisées.

Grâce à ses fonctionnalités avancées, Solidity permet de gérer les permissions, de contrôler les accès aux données et d'assurer la transparence des opérations. Il est ainsi largement utilisé pour le développement d'applications décentralisées dans des domaines sensibles tels que la finance et la santé.

Rôle des Smart Contracts dans le projet

Dans le cadre de ce projet, les smart contracts développés en Solidity assurent la gestion sécurisée et décentralisée des dossiers médicaux. Ils permettent d'enregistrer les informations médicales des patients sur la Blockchain tout en garantissant l'intégrité et la traçabilité des données. Chaque opération, telle que l'ajout ou la consultation d'un dossier médical, est enregistrée de manière permanente, ce qui renforce la transparence du système.

Les smart contracts jouent également un rôle essentiel dans la gestion des autorisations d'accès. Ils définissent les règles permettant aux médecins et aux patients d'accéder aux données médicales selon des droits bien précis. Cette approche offre aux patients un meilleur contrôle sur leurs informations personnelles et limite les risques d'accès non autorisés.

Enfin, l'utilisation des smart contracts permet d'automatiser plusieurs processus du système, réduisant ainsi les interventions manuelles et les risques d'erreurs. Dans ce projet, ils constituent le lien principal entre l'interface web et la Blockchain, assurant une communication fiable et sécurisée au sein de la plateforme de suivi des dossiers médicaux.

2.3 Truffle & Ganache

Le développement d'une application basée sur la blockchain nécessite des outils permettant de créer, tester et déployer les smart contracts de manière efficace. Dans ce projet, les frameworks Truffle et Ganache ont été utilisés afin de faciliter le cycle complet de développement des contrats intelligents sur Ethereum.

Truffle Framework

Truffle est un framework de développement dédié aux applications blockchain basées sur Ethereum. Il fournit un ensemble d'outils permettant de simplifier la création, la compilation, le déploiement et le test des smart contracts écrits en Solidity.

Parmi les principales fonctionnalités offertes par Truffle, on peut citer :

- La compilation automatique des smart contracts Solidity.
- La gestion des migrations pour le déploiement des contrats sur différents réseaux.
- Un environnement de test intégré basé sur JavaScript.
- Une structure de projet organisée facilitant la maintenance du code.

Dans le cadre de ce projet, Truffle a été utilisé pour structurer le développement des smart contracts liés à la gestion des dossiers médicaux, assurer leur compilation correcte et automatiser leur déploiement sur un réseau Ethereum local.

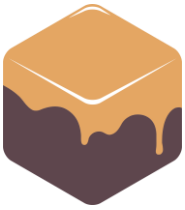
Ganache CLI

Ganache CLI est un outil permettant de simuler un réseau blockchain Ethereum en local. Il offre un environnement de test sécurisé et rapide, idéal pour le développement et le débogage des smart contracts avant leur déploiement sur un réseau réel.

Ganache fournit notamment :

- Des comptes Ethereum préconfigurés avec des soldes en Ether fictif.
- Un réseau local instantané avec des blocs générés automatiquement.
- Un suivi détaillé des transactions, des appels aux contrats et de la consommation de gas.

Dans ce projet, Ganache CLI a été utilisé comme blockchain locale afin de tester les interactions entre les smart contracts, l'interface web et le backend, sans dépendre d'un réseau public.



Déploiement local des contrats

Le déploiement local des smart contracts constitue une étape essentielle du processus de développement. Grâce à l'intégration de Truffle et Ganache, les contrats intelligents ont été déployés sur un réseau Ethereum simulé en local.

Le processus de déploiement se déroule selon les étapes suivantes :

- Compilation des smart contracts à l'aide de Truffle.
- Configuration du réseau local Ganache dans le fichier de configuration Truffle.
- Exécution des scripts de migration pour déployer les contrats sur la blockchain locale.
- Récupération de l'adresse du contrat déployé et de son ABI pour permettre l'interaction avec le frontend via Web3.js.

Cette approche permet de tester l'ensemble du système (smart contracts, backend Flask et interface web) dans un environnement contrôlé, garantissant ainsi la fiabilité et la sécurité avant toute mise en production.

2.4 IPFS (InterPlanetary File System)

IPFS (InterPlanetary File System) est un système de stockage et de partage de fichiers décentralisé, conçu pour pallier les limites des systèmes de stockage centralisés traditionnels. Contrairement aux architectures classiques basées sur des serveurs uniques, IPFS repose sur un réseau pair-à-pair (Peer-to-Peer) dans lequel les fichiers sont distribués sur plusieurs nœuds du réseau.

Le principe fondamental d'IPFS repose sur l'adressage par contenu. Chaque fichier stocké sur le réseau est identifié par une empreinte cryptographique unique (hash), appelée CID (Content Identifier). Ce mécanisme garantit l'intégrité des données, car toute modification du contenu entraîne automatiquement un changement de son identifiant.

Dans le domaine médical, l'utilisation d'IPFS présente un avantage majeur pour le stockage des fichiers volumineux tels que les rapports médicaux, les images médicales ou les documents cliniques. Stocker directement ces données sur la blockchain serait coûteux et inefficace en raison des limitations de taille et des frais de gas.

Dans le cadre de ce projet, IPFS est utilisé comme solution de stockage décentralisée pour les fichiers médicaux, tandis que la blockchain conserve uniquement les références cryptographiques (hashs IPFS) associées à ces fichiers. Cette approche hybride permet de combiner la sécurité, la traçabilité et l'immutabilité de la blockchain avec l'efficacité et la flexibilité du stockage distribué offert par IPFS.

Ainsi, l'intégration d'IPFS contribue à améliorer la scalabilité du système tout en garantissant la confidentialité et l'intégrité des données médicales, répondant aux exigences des systèmes de santé modernes.

2.5 Intelligence Artificielle

L'Intelligence Artificielle (IA) occupe une place centrale dans les systèmes informatiques modernes, notamment dans le domaine médical. Elle permet d'analyser de grandes quantités de données, d'identifier des schémas complexes et d'assister les professionnels de santé dans la prise de décision. Dans ce projet, l'IA est utilisée pour la détection automatique des situations médicales potentiellement critiques à partir des dossiers médicaux.

Définition de l'Intelligence Artificielle

L'Intelligence Artificielle désigne l'ensemble des techniques informatiques visant à créer des systèmes capables de simuler certaines capacités humaines telles que l'apprentissage, le raisonnement, la perception et la prise de décision.

Dans le domaine médical, l'IA est utilisée pour : L'analyse automatique des données cliniques,L'aide au diagnostic,La prédiction de l'évolution des maladies,La détection précoce des anomalies et des cas critiques.

Grâce à ces capacités, l'IA contribue à améliorer la qualité des soins, à réduire les erreurs humaines et à optimiser le temps des professionnels de santé.

Machine Learning

Le Machine Learning (apprentissage automatique) est une sous-branche de l'Intelligence Artificielle qui permet aux systèmes informatiques d'apprendre à partir des données, sans être explicitement programmés pour chaque tâche.

Le principe du Machine Learning repose sur :La collecte de données pertinentes,L'entraînement d'un modèle sur ces données,L'évaluation des performances du modèle,L'utilisation du modèle pour effectuer des prédictions sur de nouvelles données.

Dans le cadre de ce projet, le Machine Learning est utilisé pour analyser les informations contenues dans les dossiers médicaux (diagnostics, symptômes, âge, sexe, etc.) afin d'identifier automatiquement les patients présentant des risques médicaux élevés.

Modèles de classification

Les modèles de classification sont des algorithmes de Machine Learning dont l'objectif est d'attribuer une catégorie ou une classe à une donnée. En contexte médical, ces modèles sont particulièrement utiles pour distinguer entre des cas normaux et des cas potentiellement critiques.

Parmi les modèles de classification couramment utilisés, on retrouve :La régression logistique,Les arbres de décision,Les forêts aléatoires (Random Forest),Les machines à vecteurs de support (SVM).

Dans ce projet, un modèle de classification est utilisé pour analyser les dossiers médicaux et déterminer si un patient présente une situation critique nécessitant une attention médicale particulière. Les résultats fournis par le modèle sont ensuite exploités par le système afin d'assister le médecin dans la prise de décision, tout en conservant un rôle de supervision humaine.

2.6 Flask

Présentation de Flask

Flask est un framework web léger développé en langage Python. Il est classé parmi les micro-frameworks, car il fournit les fonctionnalités essentielles pour le développement d'applications web sans imposer une structure complexe ou des dépendances lourdes. Flask se distingue par sa simplicité, sa flexibilité et sa facilité d'intégration avec d'autres bibliothèques Python, notamment celles utilisées en intelligence artificielle et en science des données.

Grâce à son architecture modulaire, Flask permet de créer des applications web évolutives et bien structurées. Il est largement utilisé pour le développement d'API RESTful, ce qui en fait un choix approprié pour les systèmes distribués et les architectures modernes orientées services.

Rôle de Flask dans le projet

Dans le cadre de ce projet, Flask joue un rôle central en tant que couche intermédiaire entre l'interface utilisateur, les modèles d'intelligence artificielle et la blockchain. Il assure la communication entre le frontend et les composants backend du système.

Flask permet de recevoir les données saisies par le médecin ou le patient depuis l'interface web, de les traiter, puis de les transmettre soit aux modèles d'intelligence artificielle pour l'analyse des cas médicaux, soit à la blockchain pour l'enregistrement sécurisé des dossiers médicaux. Il facilite également l'exposition de services permettant la récupération des résultats d'analyse et leur affichage sur l'interface utilisateur. Ainsi, Flask contribue à l'orchestration globale du système et garantit une interaction fluide entre ses différents modules.

2.7 Web Technologies

HTML et CSS

HTML et CSS constituent les technologies de base pour la création des interfaces web. HTML permet de structurer le contenu des pages web, tandis que CSS est utilisé pour la mise en forme et la présentation visuelle de ces pages. Ensemble, ces technologies assurent la création d'interfaces claires, ergonomiques et adaptées aux besoins des utilisateurs.

Dans ce projet, HTML et CSS sont utilisés pour concevoir les interfaces destinées aux médecins et aux patients. L'objectif principal est de proposer une interface simple, intuitive et accessible, facilitant la consultation et la gestion des dossiers médicaux tout en améliorant l'expérience utilisateur.

JavaScript

JavaScript est un langage de programmation côté client permettant de rendre les pages web interactives et dynamiques. Il joue un rôle essentiel dans la gestion des événements, la validation des formulaires et la communication asynchrone avec le serveur.

Dans le contexte de ce projet, JavaScript est utilisé pour assurer l'interaction entre l'utilisateur et l'application, ainsi que pour l'échange de données avec le backend Flask et la blockchain. Il permet notamment d'envoyer des requêtes, de traiter les réponses reçues et de mettre à jour l'interface sans nécessiter un rechargement complet de la page.

Web3.js

Web3.js est une bibliothèque JavaScript permettant aux applications web d'interagir avec la blockchain Ethereum. Elle offre des fonctionnalités pour communiquer avec les smart contracts déployés, lire les données stockées sur la blockchain et exécuter des transactions de manière sécurisée.

Dans ce projet, Web3.js est utilisée pour établir la connexion entre l'interface web et les smart contracts. Elle permet aux utilisateurs d'interagir avec la blockchain à travers leurs portefeuilles numériques, d'enregistrer les dossiers médicaux de manière décentralisée et de consulter les informations stockées tout en garantissant la transparence, la traçabilité et la sécurité des données.

MetaMask : Le pont entre l'utilisateur et la Blockchain

MetaMask est utilisé dans ce projet comme une extension de navigateur agissant en tant que portefeuille numérique (Wallet) et passerelle vers le réseau Ethereum (ou Ganache). Son rôle ne se limite pas au stockage de l'Ether, mais englobe plusieurs fonctions critiques pour le système :

- Gestion de l'Identité Numérique : Il permet d'identifier de manière unique chaque acteur du système (médecin ou administrateur) via son adresse publique Ethereum.
- Signature des Transactions : Chaque action d'écriture sur la blockchain, comme l'enregistrement d'un diagnostic médical, nécessite une signature cryptographique sécurisée fournie par MetaMask pour garantir l'authenticité de l'expéditeur.
- Interaction avec les Smart Contracts : Il sert d'interface entre le navigateur web et les contrats intelligents déployés, facilitant l'exécution des fonctions storeMe-

dicalData et getPatientData sans nécessiter de nœud Ethereum complet côté client.

— Gestion du Gas : Il offre une interface de contrôle pour estimer et valider les frais de transaction (Gas) nécessaires au traitement des données sur le registre distribué.

2.8 Conclusion

Ce chapitre a permis de présenter les technologies et outils utilisés pour la réalisation du système proposé. Il a mis en évidence le rôle central de la Blockchain Ethereum et des smart contracts dans la sécurisation et la traçabilité des dossiers médicaux, ainsi que l'apport d'IPFS pour le stockage décentralisé des fichiers médicaux. L'intégration de l'Intelligence Artificielle, à travers le Machine Learning, permet quant à elle la détection automatique des cas critiques, tandis que Flask, Web3.js et les technologies web assurent la communication et l'interaction entre les différents composants du système. Ces choix technologiques constituent une base solide pour la conception et l'implémentation de la solution, qui seront détaillées dans le chapitre suivant.

Chapitre 3

Analyse et Conception du Système

Ce chapitre est consacré à l'analyse des besoins et à la conception du système proposé. Il vise à identifier les acteurs, les besoins fonctionnels et non fonctionnels, puis à définir l'architecture globale de la solution. Ce travail de conception permet de traduire les exigences du domaine médical en une architecture technique cohérente, sécurisée et adaptée à une gestion décentralisée et intelligente des dossiers médicaux.

3.1 Analyse des besoins

L'analyse des besoins constitue une étape essentielle dans la conception de tout système informatique, car elle permet d'identifier précisément les attentes des utilisateurs ainsi que les fonctionnalités que le système doit offrir. Dans le cadre de ce projet, cette analyse vise à définir les acteurs impliqués dans le système de suivi des dossiers médicaux décentralisés, ainsi que leurs rôles et interactions avec la plateforme proposée.

3.1.1 Acteurs du système

Le système de suivi des dossiers médicaux décentralisés repose sur trois acteurs principaux qui interagissent avec la plateforme. Chaque acteur dispose de rôles, de responsabilités et de droits d'accès spécifiques, définis afin d'assurer une gestion sécurisée, cohérente et fiable des données médicales.

L'Administrateur (Admin)

L'administrateur représente l'acteur responsable de la gestion globale du système. Il joue un rôle essentiel dans la configuration, le contrôle et la supervision de la plateforme. Contrairement au médecin et au patient, l'administrateur n'intervient pas directement dans le contenu médical des dossiers, mais veille au bon fonctionnement et à la sécurité de l'ensemble du système.

Dans le cadre du projet, l'administrateur est chargé de la gestion des comptes et des rôles des utilisateurs, notamment l'enregistrement et l'autorisation des médecins au sein de la plateforme. Il assure également le contrôle des accès afin de garantir que seules les

entités autorisées puissent interagir avec le smart contract et les données stockées sur la blockchain.

De plus, l'administrateur joue un rôle de supervision en s'assurant du respect des règles de sécurité et des permissions définies dans le système. Cette séparation claire des responsabilités permet de renforcer la fiabilité du système et d'éviter toute utilisation abusive ou non autorisée des données médicales.

Le Médecin

Le médecin est l'acteur principal du système en ce qui concerne la gestion opérationnelle des dossiers médicaux. Il intervient directement dans la création, la mise à jour et le suivi des informations médicales des patients. Son rôle est central, car la qualité et la pertinence des données enregistrées dépendent de son expertise médicale.

Dans le système proposé, le médecin est autorisé à ajouter de nouveaux dossiers médicaux, à renseigner les diagnostics, les traitements et les observations cliniques, ainsi qu'à consulter l'historique médical des patients. Ces actions sont enregistrées de manière sécurisée et traçable grâce à la blockchain.

Par ailleurs, le médecin interagit avec le module d'intelligence artificielle intégré à la plateforme. Ce module permet d'analyser les données médicales et de détecter automatiquement des situations potentiellement critiques. Le système agit ainsi comme un outil d'aide à la décision médicale, tout en laissant la décision finale au médecin.

Le Patient

Le patient est l'acteur directement concerné par les données médicales stockées dans le système. Il est au centre de la plateforme, puisque l'ensemble des informations gérées concernent son état de santé et son suivi médical.

Dans ce projet, le patient dispose principalement d'un droit de consultation de son dossier médical. Il peut accéder à ses informations de santé, consulter l'historique de ses dossiers et suivre l'évolution de son état médical. Cette fonctionnalité permet de renforcer la transparence et d'impliquer davantage le patient dans son parcours de soins.

Le système garantit également que les données du patient sont protégées contre tout accès non autorisé. Les mécanismes de sécurité mis en place assurent la confidentialité et l'intégrité des informations, tout en permettant au patient de conserver un certain contrôle sur l'accès à ses données personnelles.

3.1.2 Besoins fonctionnels

Les besoins fonctionnels décrivent l'ensemble des fonctionnalités que le système doit assurer afin de répondre aux attentes des utilisateurs et de garantir le bon déroulement du processus de gestion des dossiers médicaux. Ces besoins définissent les interactions principales entre les acteurs du système et la plateforme proposée.

L'une des fonctionnalités essentielles du système concerne l'ajout des dossiers médicaux. Le système doit permettre au médecin autorisé de créer et d'enregistrer de

nouveaux dossiers médicaux pour les patients. Ces dossiers incluent des informations telles que les données cliniques, les diagnostics, les traitements prescrits et les observations médicales. L'enregistrement des dossiers doit être sécurisé et effectué de manière immuable grâce à l'utilisation de la blockchain, garantissant ainsi l'intégrité et la traçabilité des données.

La consultation des dossiers médicaux constitue également un besoin fonctionnel fondamental. Le médecin doit pouvoir accéder à l'historique médical des patients afin d'assurer un suivi continu et cohérent de leur état de santé. De son côté, le patient doit pouvoir consulter son propre dossier médical, ce qui renforce la transparence et favorise son implication dans le processus de soins. L'accès aux informations est strictement contrôlé selon le rôle de chaque acteur afin de préserver la confidentialité des données.

Un autre besoin fonctionnel important du système est la détection automatique des cas critiques. Grâce à l'intégration d'un module d'intelligence artificielle, le système doit être capable d'analyser les données médicales enregistrées et d'identifier les situations présentant un risque potentiel pour la santé du patient. Cette fonctionnalité permet d'alerter le médecin et de faciliter une prise de décision rapide et appropriée.

Enfin, la sécurité des accès représente un besoin fonctionnel majeur. Le système doit garantir que seules les entités autorisées peuvent accéder aux fonctionnalités et aux données médicales. Cette sécurité repose sur des mécanismes d'authentification, de gestion des rôles et de permissions, ainsi que sur les propriétés de la blockchain, afin de prévenir tout accès non autorisé ou toute manipulation frauduleuse des informations.

3.1.3 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes et les critères de qualité que le système doit respecter afin d'assurer son efficacité, sa fiabilité et son acceptabilité dans un environnement médical sensible. Contrairement aux besoins fonctionnels, ils ne concernent pas directement les fonctionnalités offertes, mais plutôt la manière dont le système doit fonctionner.

La sécurité constitue l'un des besoins non fonctionnels les plus critiques du système. Les données médicales étant hautement sensibles, le système doit garantir leur confidentialité, leur intégrité et leur protection contre les accès non autorisés. L'utilisation de la blockchain permet d'assurer l'immuabilité des données, tandis que des mécanismes de contrôle d'accès et de chiffrement renforcent la protection des informations médicales.

La performance représente également un critère essentiel, notamment dans un contexte médical où la rapidité d'accès à l'information peut être déterminante. Le système doit être capable de répondre efficacement aux requêtes des utilisateurs, que ce soit pour l'ajout, la consultation ou l'analyse des dossiers médicaux. Les temps de réponse doivent rester acceptables afin de ne pas perturber le travail des professionnels de santé.

Enfin, la fiabilité du système est un besoin non fonctionnel fondamental. La plateforme doit garantir une disponibilité continue et un fonctionnement stable, même en cas de charge élevée ou de défaillance partielle de certains composants. La nature décentralisée de la blockchain contribue à renforcer cette fiabilité en réduisant les risques liés à un point de défaillance unique. Un système fiable est indispensable pour assurer la continuité des soins et maintenir la confiance des utilisateurs.

3.2 Architecture globale du système

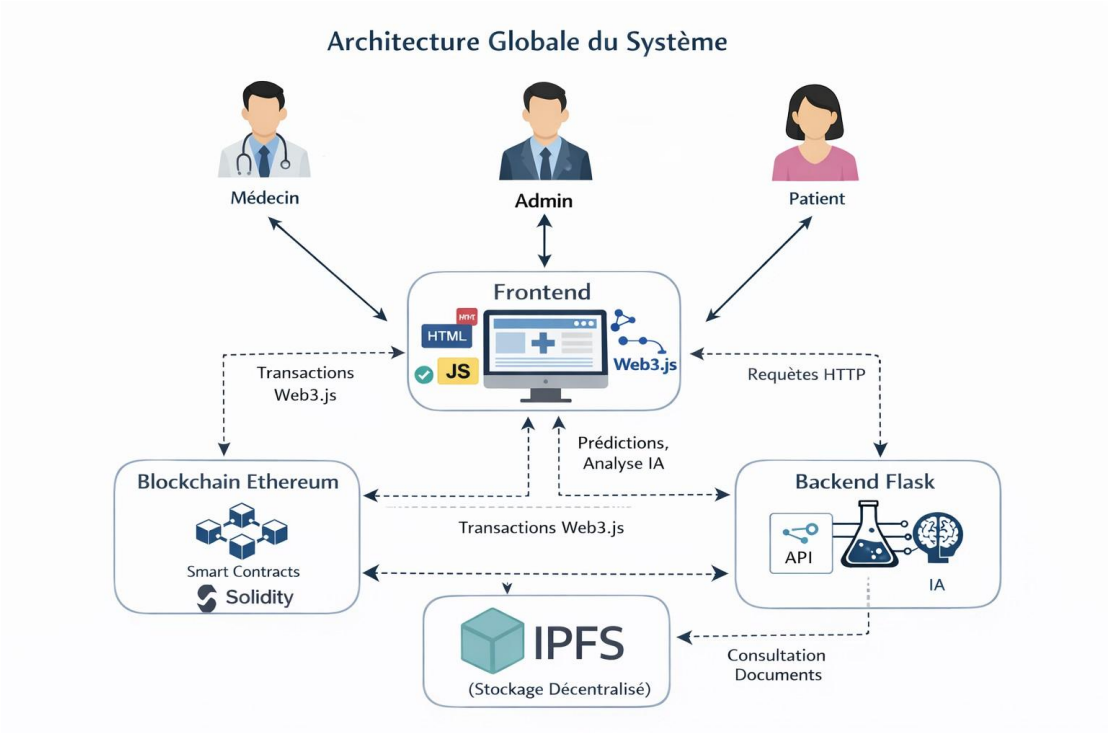


Figure 3.1. architecture globale du système.

L'architecture globale du système proposé repose sur une approche modulaire, distribuée et décentralisée, visant à assurer la sécurité, la confidentialité, la traçabilité et la performance dans la gestion des dossiers médicaux. Cette architecture est structurée autour de quatre composants principaux : le Frontend, le Backend Flask, la Blockchain Ethereum et le système de stockage décentralisé IPFS. Cette séparation claire des responsabilités permet une meilleure maintenabilité du système, une évolutivité accrue et une communication efficace entre les différents modules.

Frontend (Interface utilisateur)

Le Frontend représente la couche de présentation du système. Il constitue le point d'interaction principal entre les acteurs du système, à savoir l'administrateur, le médecin et le patient, et la plateforme. À travers une interface web développée à l'aide de HTML, CSS et JavaScript, les utilisateurs peuvent effectuer plusieurs actions telles que l'ajout et la consultation des dossiers médicaux, la visualisation des documents médicaux ainsi que l'affichage des résultats issus de l'analyse par intelligence artificielle.

Le Frontend assure également l'interaction directe avec la blockchain grâce à la bibliothèque Web3.js, permettant l'envoi de transactions vers les smart contracts et la lecture des données stockées sur la blockchain Ethereum. En parallèle, il communique avec le Backend Flask via des requêtes HTTP (API REST) pour l'analyse intelligente et le traitement des données médicales.

Backend Flask (Serveur applicatif et IA)

Le Backend, basé sur le framework Flask, joue le rôle de serveur applicatif intermédiaire entre le Frontend, les modèles d'Intelligence Artificielle et le stockage décentralisé IPFS. Il est responsable du traitement des requêtes envoyées par le Frontend, notamment celles liées à l'analyse des données médicales.

Ce serveur intègre des modèles de Machine Learning préalablement entraînés afin de détecter automatiquement les cas potentiellement critiques à partir des données médicales fournies. Une fois l'analyse effectuée, les résultats de prédiction sont renvoyés au Frontend pour être affichés au médecin.

Le Backend Flask assure également la communication avec IPFS, permettant l'envoi et la récupération des documents médicaux (rapports, images, analyses). Seuls les hashs IPFS sont ensuite stockés sur la blockchain afin de garantir l'intégrité des documents sans alourdir la chaîne.

Blockchain Ethereum (Smart Contracts)

La Blockchain Ethereum constitue le noyau de sécurité et de traçabilité du système. Elle repose sur des smart contracts développés en Solidity, déployés sur un réseau Ethereum local (Ganache). Ces smart contracts assurent la gestion des patients, des médecins, des dossiers médicaux ainsi que des règles de permissions et d'accès.

Toutes les opérations sensibles, telles que l'ajout ou la consultation d'un dossier médical, sont enregistrées sous forme de transactions immuables, garantissant ainsi l'intégrité, la transparence et la traçabilité des données. La blockchain permet également d'éliminer le point de défaillance unique et de renforcer la confiance entre les différents acteurs du système.

IPFS (Stockage décentralisé)

Le système IPFS (InterPlanetary File System) est utilisé comme solution de stockage décentralisé pour les documents médicaux volumineux. Contrairement à la blockchain, qui n'est pas adaptée au stockage de grandes quantités de données, IPFS permet de stocker les fichiers de manière distribuée tout en garantissant leur intégrité grâce au mécanisme de hash cryptographique.

Les documents médicaux sont stockés sur IPFS, et leur hash unique est enregistré sur la blockchain via les smart contracts. Cette approche hybride permet de combiner les avantages du stockage décentralisé et de la blockchain tout en optimisant les performances et les coûts.

Communication entre les composants

La communication entre les différents composants du système est assurée de manière sécurisée et cohérente :

— Le Frontend communique avec la Blockchain via Web3.js.
— Le Frontend échange avec le Backend Flask à travers des API REST.

— Le Backend Flask interagit avec IPFS pour la gestion des documents.
— La Blockchain stocke uniquement les métadonnées et les hash IPFS.

Cette architecture globale permet ainsi de combiner efficacement :

— la sécurité et la traçabilité de la blockchain,
— la puissance d'analyse de l'Intelligence Artificielle,
— la flexibilité du stockage décentralisé IPFS,
— et la simplicité d'utilisation d'une interface web moderne,

répondant ainsi aux exigences d'un système de suivi médical fiable, sécurisé et performant.

3.3 Modélisation UML

Les diagrammes UML (*Unified Modeling Language*) constituent un outil essentiel pour la modélisation et la représentation graphique du fonctionnement du système. Ils permettent de visualiser les interactions entre les acteurs, les composants du système ainsi que le déroulement des différents processus. Dans le cadre de ce projet, les diagrammes UML facilitent la compréhension globale de l'architecture et du comportement du système de suivi des dossiers médicaux décentralisés.

3.3.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présente une vue globale des fonctionnalités offertes aux acteurs. Il met en évidence les interactions principales : gestion des médecins (admin), gestion des patients et mises à jour (médecin), consultation (patient).

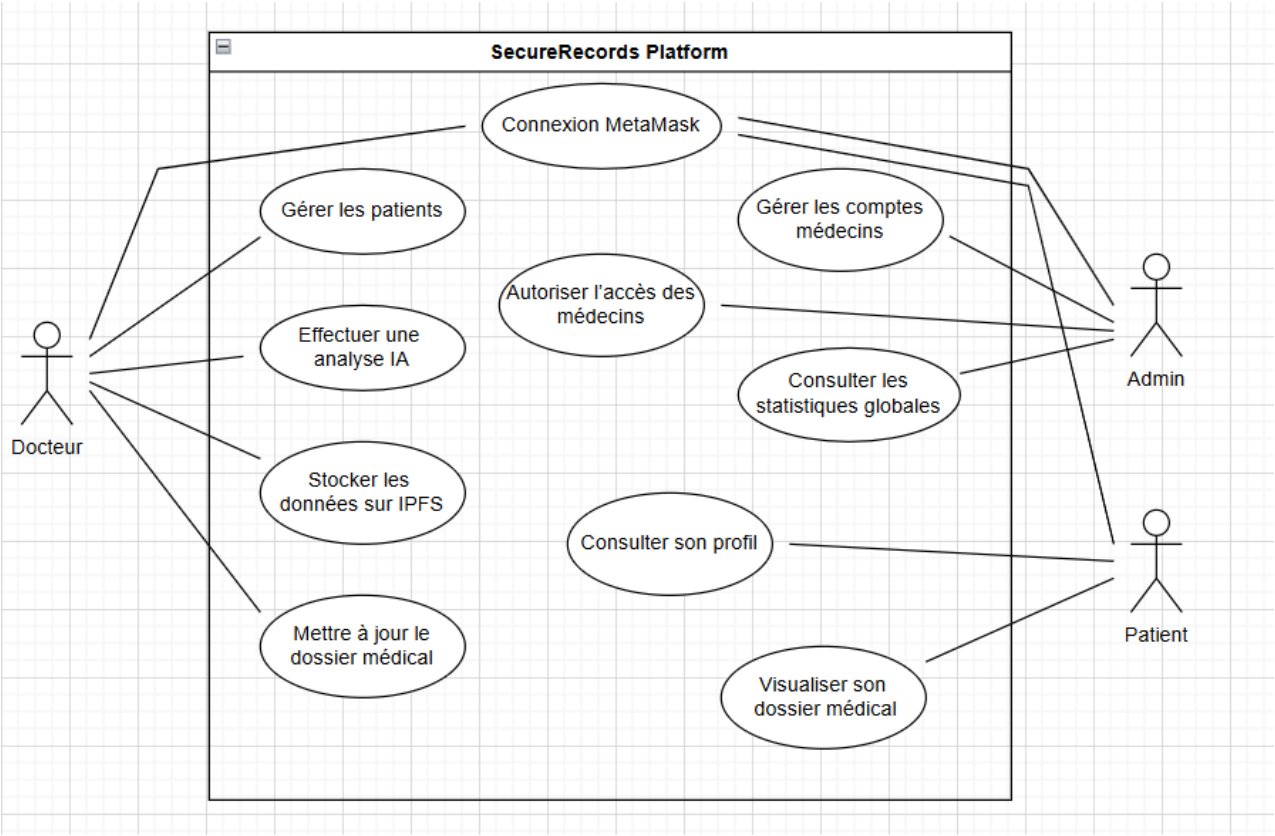


Figure 3.2. Diagramme global de cas d'utilisation.

3.3.2 Diagrammes de séquence

Le diagramme de séquence décrit le déroulement temporel des interactions entre les différents composants du système. Il met en évidence l'ordre des échanges de messages entre les acteurs, le Frontend, la Blockchain et le serveur d'Intelligence Artificielle.

Dans le contexte de ce projet, ce diagramme illustre par exemple le processus d'ajout d'un dossier médical ou la détection d'un cas critique. L'acteur initie une action via le Frontend, lequel transmet les données soit au smart contract sur la blockchain pour l'enregistrement sécurisé, soit au serveur Flask pour l'analyse intelligente. Les résultats sont ensuite renvoyés vers le Frontend pour être affichés à l'utilisateur.

Ce diagramme permet de mieux comprendre la logique de communication entre les différents modules et d'assurer la cohérence des échanges au sein du système.

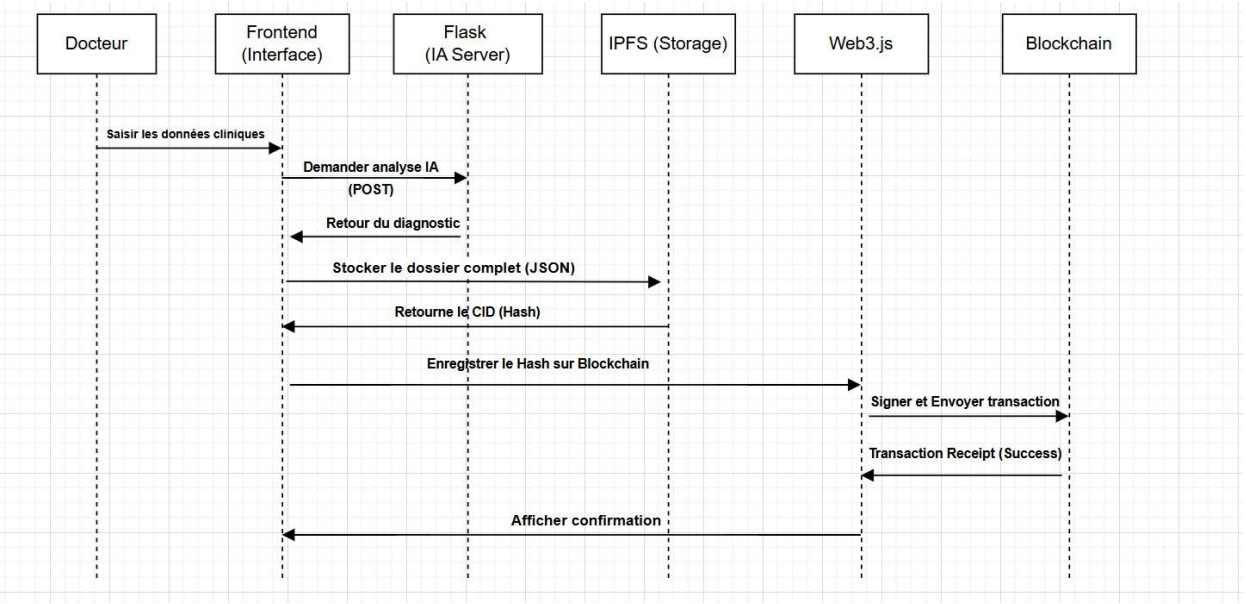


Figure 3.3. Diagramme de séquence : mise à jour du dossier (médecin).

Scénario 1 : mise à jour du dossier par le médecin (IA + IPFS + Blockchain)

Scénario nominal (aligné avec l'implémentation)

- **Saisie des données :** le médecin remplit le formulaire clinique sur l'interface Web3.
- **Analyse IA :** le front-end envoie une requête POST au serveur Flask (/predict_risk) avec les paramètres cliniques.
- **Retour du diagnostic :** le serveur applique le prétraitement (scaler) et le modèle ML, puis renvoie une prédiction (risque faible/élevé).
- **Construction du JSON :** le front-end regroupe les champs saisis et le résultat IA dans un objet JSON (dossier).
- **Stockage sur IPFS :** le JSON est envoyé au nœud IPFS via l'API HTTP ; IPFS retourne un **CID**.
- **Ancrage on-chain :** le front-end appelle la fonction de mise à jour du hash/CID du patient (ex. updatePatientDataHash) via Web3.js.
- **Signature :** MetaMask demande au médecin de signer la transaction.
- **Confirmation :** la blockchain valide la transaction et retourne un reçu ; le contrat enregistre le CID (état courant + historique) et émet un événement.
- **Finalisation :** l'interface affiche un message de succès et met à jour l'affichage du dossier.

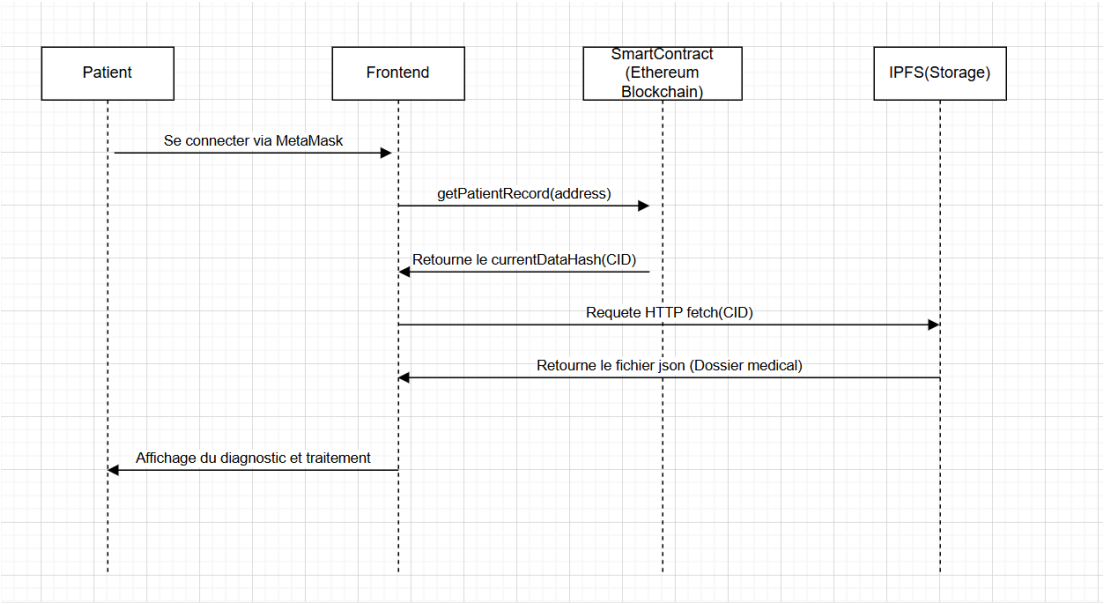


Figure 3.4. Diagramme de séquence : consultation du dossier par le patient.

Scénario 2 : consultation du dossier par le patient (lecture)

Pour compléter l'analyse (niveau Master), un second scénario est essentiel : la consultation côté patient.

- Le patient se connecte via MetaMask (sélection de l'adresse du patient).
- Le front-end appelle une fonction de lecture du smart contract pour récupérer le **CID courant** (et éventuellement l'historique).
- Le front-end résout le CID via une passerelle IPFS et affiche le contenu du JSON.

Ce scénario illustre la séparation *on-chain* (CID) / *off-chain* (contenu) et la vérifiabilité du dossier.

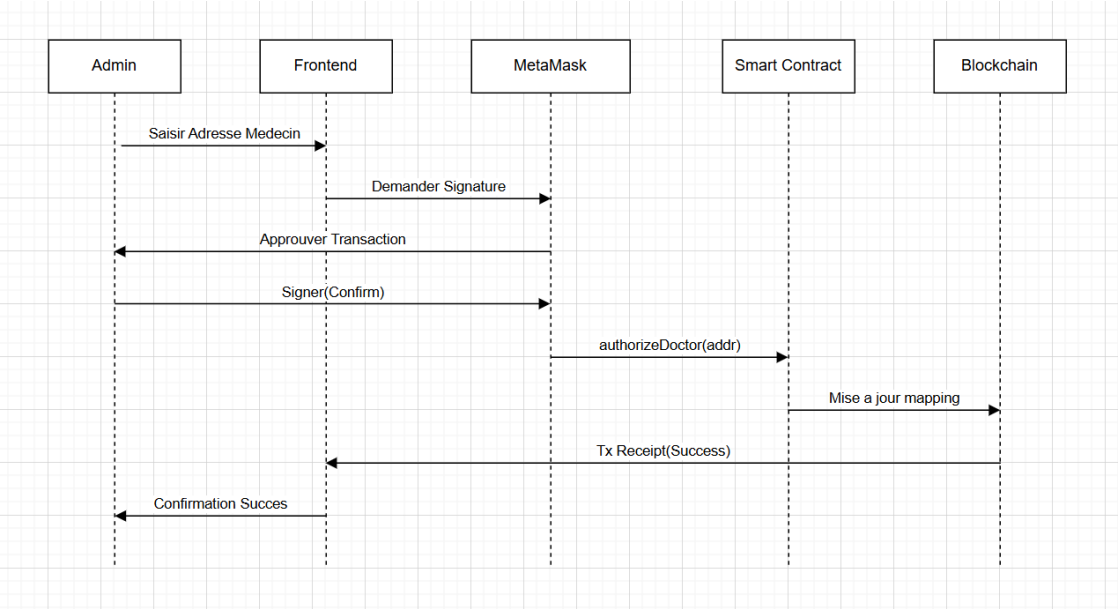


Figure 3.5. autorisation d'un médecin par l'administrateur.

Scénario 3 : autorisation d'un médecin par l'administrateur

Un troisième scénario clarifie la gouvernance :

— L'admin se connecte via MetaMask (adresse propriétaire admin).
— Le front-end admin appelle une fonction du smart contract du type authorizeDoctor(address).
— MetaMask signe la transaction ; la blockchain enregistre l'autorisation (mise à jour de authorizedDoctors).

Ce scénario explicite le mécanisme de confiance et la séparation des responsabilités.

3.4 Conclusion

À travers ce chapitre, une analyse détaillée des besoins du système a été réalisée, mettant en évidence les exigences fonctionnelles et non fonctionnelles liées à la gestion sécurisée des dossiers médicaux. La conception de l'architecture globale a permis de définir une solution modulaire et décentralisée intégrant le Frontend, le Backend Flask, la Blockchain Ethereum et le stockage IPFS. Les diagrammes UML ont également contribué à clarifier le fonctionnement du système et les interactions entre ses différents composants. Cette phase de conception constitue ainsi une base solide pour l'implémentation technique présentée dans le chapitre suivant.

Chapitre 4

Implémentation de la Solution

Ce chapitre est dédié à l'implémentation technique de la solution proposée. Il présente en détail la mise en œuvre du smart contract pour la gestion des patients et des dossiers médicaux, le déploiement sur la blockchain à l'aide de Truffle, ainsi que l'intégration du stockage décentralisé via IPFS. Il aborde également l'implémentation du module d'Intelligence Artificielle, de l'API Flask et du Frontend, permettant ainsi d'assurer une solution complète, sécurisée et fonctionnelle.

4.1 Implémentation de la Blockchain

L'implémentation de la blockchain constitue un élément central de la solution proposée, puisqu'elle assure le stockage sécurisé, la traçabilité et l'intégrité des dossiers médicaux. Dans ce projet, la technologie blockchain est exploitée à travers un smart contract déployé sur un réseau Ethereum local. Ce choix permet de garantir une gestion décentralisée des données médicales tout en contrôlant les accès selon des règles bien définies.

Le smart contract agit comme une couche de confiance entre les différents acteurs du système et remplace les mécanismes classiques de gestion centralisée des bases de données. Il définit l'ensemble des structures de données, des fonctions métiers et des règles de sécurité nécessaires au bon fonctionnement de la plateforme.

Structure du Smart Contract MedicalRecords.sol

Le smart contract MedicalRecords.sol représente le cœur de la couche blockchain du système. Il est développé en langage Solidity et est conçu pour gérer les patients, les dossiers médicaux ainsi que les droits d'accès associés. Sa structure repose sur une organisation claire des données et des fonctions afin de garantir la lisibilité, la maintenabilité et la sécurité du code.

Le contrat contient plusieurs structures de données permettant de représenter les entités principales du système. Ces structures incluent notamment les informations relatives aux patients ainsi que les dossiers médicaux associés à chacun d'eux. Chaque dossier médical est lié à un identifiant unique et peut contenir des références vers des données stockées hors chaîne, notamment sous forme de CID IPFS, afin d'éviter le

stockage direct de données volumineuses sur la blockchain.

Le smart contract intègre également des mappings permettant d'associer chaque patient à ses dossiers médicaux et de gérer efficacement les relations entre les différentes entités. Cette organisation assure un accès rapide aux informations tout en respectant les principes fondamentaux de la blockchain, notamment l'immuabilité et la traçabilité.

Gestion des patients

La gestion des patients constitue une fonctionnalité essentielle du smart contract. Elle permet d'enregistrer les patients dans le système et de garantir que chaque patient est identifié de manière unique à travers son adresse blockchain. Cette approche repose sur l'utilisation des adresses Ethereum comme identifiants, ce qui renforce la sécurité et l'authenticité des utilisateurs.

Lors de l'enregistrement d'un patient, le smart contract vérifie que l'adresse utilisée n'est pas déjà associée à un patient existant, évitant ainsi les doublons et les incohérences. Les informations de base du patient sont ensuite stockées de manière sécurisée sur la blockchain, tandis que les données médicales détaillées peuvent être référencées via des liens externes.

Cette gestion décentralisée des patients permet de garantir que les données ne peuvent être ni modifiées ni supprimées de manière arbitraire, renforçant ainsi la confiance dans le système et assurant une meilleure protection des informations sensibles.

Gestion des dossiers médicaux

La gestion des dossiers médicaux est assurée directement par le smart contract à travers des fonctions dédiées à la création, à la mise à jour et à la consultation des dossiers. Chaque dossier médical est associé à un patient spécifique et ne peut être ajouté que par un médecin autorisé.

Dans le cadre du projet, les dossiers médicaux ne stockent pas directement les données cliniques complètes sur la blockchain. À la place, un identifiant de type CID (Content Identifier) issu d'IPFS est enregistré dans le smart contract. Cette approche permet de séparer le stockage des données médicales volumineuses (off-chain) de leur référence immuable (on-chain), tout en garantissant l'intégrité et la vérifiabilité des informations.

Lorsqu'un dossier est consulté, le smart contract fournit le CID correspondant, que le Frontend utilise ensuite pour récupérer les données depuis IPFS. Cette méthode assure un compromis efficace entre sécurité, performance et respect des contraintes techniques de la blockchain.

Sécurité et permissions

La sécurité et la gestion des permissions représentent un aspect fondamental de l'implémentation blockchain du système. Le smart contract définit des règles strictes d'accès afin de garantir que seules les entités autorisées peuvent effectuer certaines actions.

Dans ce projet, les fonctions critiques telles que l'ajout ou la modification des dossiers médicaux sont restreintes aux médecins autorisés. À l'inverse, les patients disposent uniquement de droits de consultation concernant leurs propres dossiers médicaux. Ces restrictions sont implémentées directement au niveau du smart contract à l'aide de conditions basées sur l'adresse de l'appelant.

Cette approche garantit que les règles de sécurité sont appliquées de manière automatique et infalsifiable, sans dépendre d'un serveur central. Grâce à l'immuabilité du smart contract, les politiques de sécurité définies lors du déploiement ne peuvent être modifiées de manière arbitraire, ce qui renforce la fiabilité globale du système.

4.2 Déploiement avec Truffle

Le déploiement du smart contract constitue une étape clé dans la mise en œuvre de la solution basée sur la blockchain. Dans ce projet, le framework Truffle est utilisé afin de faciliter le développement, le test et le déploiement des smart contracts sur un réseau Ethereum local. Truffle offre un environnement structuré qui permet d'automatiser les différentes étapes du cycle de vie du contrat, depuis la compilation jusqu'à l'interaction avec celui-ci.

Le choix de Truffle est motivé par sa compatibilité avec Solidity, sa simplicité d'utilisation et son intégration avec des outils de test et de déploiement tels que Ganache CLI. Cette approche permet de reproduire un environnement proche d'un réseau réel tout en conservant un contrôle total sur les paramètres de déploiement.

Configuration Truffle

La configuration de Truffle constitue la première étape du processus de déploiement. Elle consiste à définir l'environnement de travail ainsi que les paramètres nécessaires à la compilation et à l'exécution des smart contracts. Cette configuration est principalement réalisée à travers le fichier truffle-config.js, qui permet de spécifier les réseaux cibles, la version du compilateur Solidity et les options associées.

Dans le cadre de ce projet, Truffle est configuré pour se connecter à un réseau Ethereum local fourni par Ganache CLI. Ce réseau local simule le comportement d'une blockchain réelle et fournit plusieurs comptes préconfigurés, facilitant ainsi les tests et le déploiement. La version du compilateur Solidity est choisie de manière cohérente avec celle utilisée dans le développement du smart contract, garantissant ainsi la compatibilité et la stabilité du code.

Cette phase de configuration permet d'assurer que le processus de déploiement se déroule dans un environnement contrôlé, reproductible et conforme aux exigences du projet.

Migration du contrat

La migration du smart contract correspond à l'étape durant laquelle le contrat est effectivement déployé sur la blockchain. Truffle gère cette opération à l'aide de scripts

de migration écrits en JavaScript et stockés dans le dossier dédié aux migrations.

Lors de l'exécution du processus de migration, Truffle compile d'abord le smart contract, puis le déploie sur le réseau Ethereum local en utilisant un compte fourni par Ganache. Chaque migration est versionnée, ce qui permet de suivre l'évolution du contrat et d'éviter les déploiements redondants.

Une fois le contrat déployé, une adresse unique lui est attribuée sur la blockchain. Cette adresse représente l'identifiant du contrat et est indispensable pour permettre aux autres composants du système, notamment le Frontend et l'API Flask, d'interagir avec celui-ci. La migration constitue ainsi une étape essentielle pour rendre le smart contract opérationnel et accessible aux différents modules de la plateforme.

Génération de l'ABI

Après le déploiement du smart contract, Truffle génère automatiquement une interface appelée ABI (Application Binary Interface). L'ABI décrit l'ensemble des fonctions publiques du contrat ainsi que leurs paramètres et types de retour. Elle joue un rôle fondamental dans la communication entre le smart contract et les applications externes.

Dans ce projet, l'ABI est utilisée par le Frontend, via la bibliothèque Web3.js, afin d'interagir avec le smart contract déployé. Elle permet notamment d'appeler les fonctions de lecture et d'écriture, telles que l'ajout ou la consultation des dossiers médicaux. L'ABI est également exploitée par le serveur Flask lorsque celui-ci doit accéder à certaines informations stockées sur la blockchain.

La génération automatique de l'ABI par Truffle garantit une correspondance exacte entre le smart contract et les composants applicatifs qui l'utilisent. Cette approche réduit les risques d'erreurs et facilite l'intégration entre la couche blockchain et le reste du système.

4.3 Implémentation du module d'Intelligence Artificielle

Le module d'Intelligence Artificielle constitue un élément central du système proposé, puisqu'il permet d'assurer la détection automatique des situations médicales potentiellement critiques. Ce module repose sur des techniques de Machine Learning appliquées à des données médicales structurées. Son objectif principal est d'analyser les informations des patients et de fournir une aide à la décision médicale, tout en restant complémentaire à l'expertise du médecin.

L'implémentation du module IA suit une démarche méthodologique classique comprenant la sélection du jeu de données, le prétraitement des informations, l'entraînement du modèle et enfin la sauvegarde du modèle entraîné pour une utilisation ultérieure au sein du système.

Dataset utilisé (heart_disease.csv)

Le jeu de données utilisé dans ce projet est le fichier heart_disease.csv, largement exploité dans le domaine de la recherche en santé pour la prédiction des maladies cardiovasculaires. Ce dataset contient un ensemble de données cliniques relatives à des patients, telles que l'âge, le sexe, certains indicateurs physiologiques ainsi que des informations liées à l'état cardiaque.

Le choix de ce dataset est justifié par sa pertinence par rapport à l'objectif du projet, qui vise à détecter automatiquement des cas médicaux critiques. Les données qu'il contient sont suffisamment représentatives pour entraîner un modèle de classification capable d'identifier des situations à risque. De plus, ce dataset est bien documenté et fréquemment utilisé dans des travaux académiques, ce qui renforce la crédibilité et la reproductibilité de la solution proposée.

Prétraitement des données

Avant l'entraînement du modèle de Machine Learning, une étape de prétraitement des données est indispensable afin d'améliorer la qualité et la fiabilité des résultats. Cette phase consiste principalement à nettoyer le dataset, à traiter les valeurs manquantes et à préparer les données pour leur exploitation par les algorithmes d'apprentissage automatique.

Les données numériques sont normalisées ou mises à l'échelle afin d'éviter que certaines variables n'influencent excessivement le modèle. Les attributs catégoriels, lorsqu'ils existent, sont transformés en valeurs numériques exploitables. Le dataset est ensuite divisé en deux ensembles distincts, à savoir un ensemble d'apprentissage et un ensemble de test, permettant d'évaluer les performances du modèle de manière objective.

Cette étape de prétraitement joue un rôle fondamental dans la qualité du modèle final, car elle permet de réduire le bruit dans les données et d'améliorer la capacité de généralisation du modèle.

Sauvegarde du modèle (.pkl)

Après l'entraînement et la validation du modèle, celui-ci est sauvegardé sous la forme d'un fichier .pkl. Cette sauvegarde permet de conserver le modèle entraîné afin de l'utiliser ultérieurement sans avoir à relancer tout le processus d'apprentissage.

Le modèle sauvegardé est ensuite chargé par le serveur Flask lors de l'exécution du système. Cette approche permet d'optimiser les performances globales de la plateforme en évitant des calculs coûteux en temps et en ressources. Lorsqu'une requête de prédiction est envoyée depuis le Frontend, le serveur Flask utilise directement le modèle chargé pour analyser les données médicales et retourner le résultat de la prédiction.

La sauvegarde du modèle constitue ainsi une étape essentielle pour l'intégration efficace du module d'Intelligence Artificielle au sein de l'architecture globale du système.

4.4 API Flask

Afin d'assurer la communication entre l'interface utilisateur et le module d'Intelligence Artificielle, une API REST a été mise en place à l'aide du framework Flask. Cette API joue un rôle d'intermédiaire entre le Frontend et le modèle de Machine Learning, en permettant l'envoi des données médicales à analyser et la réception des résultats de prédiction. L'utilisation de Flask se justifie par sa légèreté, sa flexibilité et sa facilité d'intégration avec des modèles d'IA développés en Python.

L'API Flask contribue ainsi à la modularité du système en isolant la logique de traitement intelligent du reste de la plateforme, tout en garantissant une communication fluide et structurée entre les différents composants.

Structure du serveur Flask

Le serveur Flask est organisé de manière simple et claire afin de faciliter la maintenance et l'évolutivité du système. Il comprend un fichier principal chargé de l'initialisation de l'application, du chargement du modèle d'Intelligence Artificielle sauvegardé et de la définition des routes de l'API.

Lors du démarrage du serveur, le modèle de Machine Learning entraîné est chargé depuis le fichier de sauvegarde au format .pkl. Cette approche permet de rendre le serveur immédiatement opérationnel pour traiter les requêtes de prédiction, sans nécessiter un nouvel entraînement du modèle. Le serveur Flask est également configuré pour accepter des requêtes provenant du Frontend, généralement sous forme de données JSON.

Cette structure garantit une séparation claire entre la logique applicative, le traitement des données et l'accès au modèle d'IA, ce qui améliore la lisibilité et la robustesse du système.

Endpoint de prédiction

L'API Flask expose un endpoint dédié à la prédiction des cas médicaux critiques. Cet endpoint reçoit les données médicales envoyées par le Frontend, les prépare dans un format compatible avec le modèle de Machine Learning, puis exécute la prédiction correspondante.

Le résultat de la prédiction indique si les données fournies correspondent à un cas normal ou à un cas potentiellement critique. Ce résultat est ensuite renvoyé au Frontend sous forme d'une réponse structurée, généralement en format JSON. Cette approche permet une intégration simple et efficace avec l'interface utilisateur, tout en assurant une communication claire et standardisée entre les modules.

L'endpoint de prédiction constitue ainsi le point central de l'interaction entre le système intelligent et les utilisateurs, en fournissant une aide à la décision rapide et automatisée.

Communication Frontend avec Intelligence Artificielle

La communication entre le Frontend et le module d'Intelligence Artificielle s'effectue exclusivement via l'API Flask. Le Frontend collecte les données nécessaires à l'analyse, soit à partir des informations saisies par le médecin, soit à partir des dossiers médicaux existants, puis les transmet au serveur Flask à travers une requête HTTP.

Une fois la requête reçue, le serveur Flask traite les données, exécute le modèle de Machine Learning et renvoie le résultat de la prédiction au Frontend. Ce dernier affiche alors l'information de manière claire et compréhensible pour l'utilisateur, notamment en signalant les cas potentiellement critiques au médecin.

Cette architecture de communication garantit une forte modularité du système, permettant d'améliorer ou de remplacer le modèle d'IA sans impacter le reste de la plateforme. Elle assure également une bonne séparation des responsabilités entre l'interface utilisateur, la logique intelligente et la gestion des données médicales, contribuant ainsi à la fiabilité et à la performance globale du système.

4.5 Développement de l'Interface Utilisateur (Frontend)

La couche Frontend de notre solution ne se limite pas à la présentation visuelle ; elle joue le rôle d'un orchestrateur central assurant la liaison entre l'utilisateur, le serveur d'intelligence artificielle, le réseau IPFS et la Blockchain Ethereum. Développée avec une architecture modulaire en JavaScript (ES6+) et Bootstrap 5, elle garantit une expérience utilisateur fluide tout en gérant la complexité des interactions décentralisées.

Interface Administrateur : Gouvernance et Contrôle d'Accès

L'interface d'administration constitue le module de gestion de la confiance (Trust Management) au sein de la plateforme. Elle permet de transposer les règles de gouvernance du Smart Contract vers une interface de gestion simplifiée :

- Gestion des Privilèges (White-listing) : Cette interface permet à l'administrateur de déclarer et d'autoriser officiellement des adresses Ethereum de médecins. En appelant la fonction authorizeDoctor, l'administrateur accorde des droits d'écriture sur la Blockchain, garantissant que seuls les praticiens certifiés peuvent injecter des données médicales.
- Gestion des Privilèges (White-listing) : Cette interface permet à l'administrateur de déclarer et d'autoriser officiellement des adresses Ethereum de médecins. En appelant la fonction authorizeDoctor, l'administrateur accorde des droits d'écriture sur la Blockchain, garantissant que seuls les praticiens certifiés peuvent injecter des données médicales.
- Audit et Surveillance du Réseau : L'administrateur peut visualiser l'état du contrat, le propriétaire actuel (Owner) et s'assurer que les paramètres de sécurité sont correctement configurés.
- Sécurisation par Signature : Chaque action de l'administrateur nécessite une signature via MetaMask, validant ainsi l'identité du gestionnaire par le biais

d'une preuve cryptographique forte avant toute modification du mapping des autorisations.

Interface Médecin : Gestion et Diagnostic

L'interface dédiée au corps médical est conçue pour optimiser le flux de travail clinique. Elle se décline en plusieurs modules fonctionnels :

- Tableau de bord (Dashboard) : Offre une vue d'ensemble sur les statistiques des patients et l'état de la connexion au réseau Blockchain.
- Module d'inscription et de saisie : Permet la collecte structurée des paramètres physiologiques. Ce module intègre des mécanismes de validation de données avant tout envoi vers l'API d'intelligence artificielle.
- Console de diagnostic : Affiche en temps réel les résultats fournis par le modèle IA, permettant au médecin de confirmer le diagnostic avant son ancrage définitif dans le dossier immuable.

Interface Patient : Transparence et Propriété des Données

L'interface patient incarne le principe de Self-Sovereign Identity (SSI). Contrairement aux systèmes traditionnels, le patient accède à ses données via son adresse publique (Wallet) :

- Authentification décentralisée : L'accès est conditionné par la possession de la clé privée associée au Wallet MetaMask, garantissant qu'aucun tiers non autorisé ne peut consulter le dossier.
- Consultation du dossier médical : Le Frontend interroge la Blockchain pour récupérer le dernier CID (Content Identifier) associé au patient, puis reconstruit visuellement le dossier à partir des données brutes récupérées sur IPFS.
- Preuve de conformité : Une fonctionnalité permet au patient de télécharger une preuve cryptographique (Hash) attestant de l'intégrité de ses données à une date donnée.

Interaction avec la Blockchain via Web3.js

L'interaction technique entre le navigateur et le Smart Contract est assurée par la bibliothèque Web3.js, qui agit comme une interface de communication avec le fournisseur Ethereum (MetaMask).

- Initialisation du Provider : Le système détecte l'injection de window.ethereum pour établir une connexion sécurisée avec le nœud Blockchain (Ganache dans notre environnement de test).
- Abstraction du Contrat (ABI) : Grâce à l'Application Binary Interface (ABI) et à l'adresse du contrat déployé, Web3.js permet d'instancier un objet JavaScript représentant le Smart Contract, rendant ses fonctions (addPatient, getPatientRecord, etc.) directement accessibles.
- Gestion des Transactions et des Appels : * Les "Sends" (Écritures) : Pour l'ajout de données, Web3.js gère la création de la transaction, le calcul des frais de gaz (Gas estimation) et la gestion de la signature par le médecin.

- Les "Calls" (Lectures) : Pour la consultation, il effectue des appels constants vers la Blockchain, ne nécessitant aucun coût en gaz, pour récupérer les informations en temps réel.

Gestion de l'état (State Management) et Expérience Utilisateur (UX)

Dans les applications décentralisées (dApps), la latence du réseau Blockchain lors de la validation des transactions représente un défi majeur pour l'expérience utilisateur. Pour pallier cela, nous avons implémenté une gestion d'état asynchrone rigoureuse :

— Indicateurs de progression (Spinners) : Afin d'éviter toute confusion lors des phases d'attente (comme le minage d'un bloc ou le chargement depuis IPFS), des indicateurs visuels dynamiques sont activés via JavaScript. Cela permet d'informer l'utilisateur que le système traite la requête en arrière-plan.

— Gestion des notifications et retours d'erreurs : Le Frontend intercepte les exceptions levées par Web3.js (rejet de signature par MetaMask, manque de fonds pour le gaz, ou erreurs de contrat). Ces erreurs sont traduites en messages compréhensibles pour l'utilisateur, garantissant la résilience de l'interface.

— Mise à jour réactive de l'interface : Une fois la confirmation de transaction (Transaction Receipt) reçue, l'état de l'interface est mis à jour automatiquement sans rechargement de la page, reflétant ainsi les nouvelles données enregistrées de manière fluide.

4.6 Conclusion

À travers ce chapitre, la réalisation concrète de la plateforme a été présentée, depuis la gestion décentralisée des patients et des dossiers médicaux jusqu'à l'intégration des technologies blockchain, IPFS et intelligence artificielle. Le déploiement du smart contract avec Truffle, la mise en place de l'API Flask et l'implémentation du module IA ont permis d'assurer une détection automatique des situations médicales critiques. Enfin, le développement du Frontend a facilité l'interaction entre les différents acteurs du système (administrateur, médecin et patient), tout en garantissant sécurité, transparence et performance. Cette implémentation valide la faisabilité et la cohérence de la solution proposée.

Chapitre 5

Tests, Résultats et Discussion

Ce chapitre présente l'évaluation complète de la plateforme développée. Il décrit les différents tests réalisés sur les Smart Contracts, le modèle d'intelligence artificielle et l'intégration globale du système. Les résultats obtenus sont ensuite analysés et discutés afin d'évaluer les performances, la fiabilité et les limites de la solution, tout en proposant des perspectives d'amélioration pour les travaux futurs.

5.1 Tests du Smart Contract

Le Smart Contract est la pierre angulaire de la confiance dans notre architecture. Étant donné l'immuabilité de la Blockchain, une phase de test rigoureuse est impérative pour garantir l'absence de failles logiques.

Tests fonctionnels

Ces tests visent à vérifier que chaque fonction du contrat MedicalRecords.sol réagit conformément aux spécifications établies. Pour ce faire, nous avons utilisé l'environnement Truffle couplé à la blockchain locale Ganache.

- Vérification de l'ajout de patient : Validation que la fonction addPatient enregistre correctement le nom, l'âge, le sexe et le CID IPFS dans le mapping global.
- Intégrité de la récupération des données : Test de la fonction getPatientRecord pour s'assurer qu'elle retourne le dernier currentDataHash exact associé à un Wallet spécifique.
- Persistance de l'état : Confirmation que les données restent immuables et accessibles après plusieurs transactions successives sur le réseau Ganache.

Tests de sécurité (Access Control)

La sécurité dans un contexte médical repose sur la gestion stricte des permissions. Nous avons testé les mécanismes de protection implémentés via les Modifiers de Solidity.

— Test du Modifier onlyOwner : Nous avons tenté d'exécuter la fonction authorizeDoctor à partir d'un compte non-administrateur. Le résultat a été un rejet systématique de la transaction (Revert), confirmant que seul le propriétaire légitime peut accorder des privilèges.
— Vérification de l'autorisation des médecins : Validation que seuls les comptes présents dans le mapping authorizedDoctors ont le droit d'écrire ou de mettre à jour un dossier médical.
— Protection contre les accès non autorisés : Test de la fonction isPatientRegistered pour garantir que le système refuse l'accès aux interfaces Dashboard si l'adresse publique n'est pas préalablement inscrite dans le contrat.

5.2 Tests du modèle IA

La fiabilité du système de détection des cas critiques repose sur la précision du modèle de classification intégré. Cette section présente les métriques de performance obtenues lors de la phase d'entraînement et de validation du modèle.

Phase d'entraînement et Sélection du modèle

Nous avons entraîné et comparé cinq modèles de Machine Learning classiquement utilisés dans la littérature médicale : Régression Logistique, K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Decision Tree (Arbre de décision) et Random Forest. L'objectif était d'identifier le modèle offrant le meilleur compromis entre précision (Accuracy) et capacité de généralisation.
L'entraînement est implémenté dans 2_AI_SERVER_FLASK/train_model.py. Le script :

— charge le dataset heart_disease.csv,
— normalise les features avec StandardScaler,
— entraîne plusieurs modèles (Logistic Regression, KNN, SVM, Decision Tree, Random Forest),
— sélectionne automatiquement le meilleur selon l'accuracy,
— sauvegarde le modèle final dans model/heart_disease_model.pkl et le scaler dans model/scaler.pkl.

Résultats de la comparaison (Accuracy)

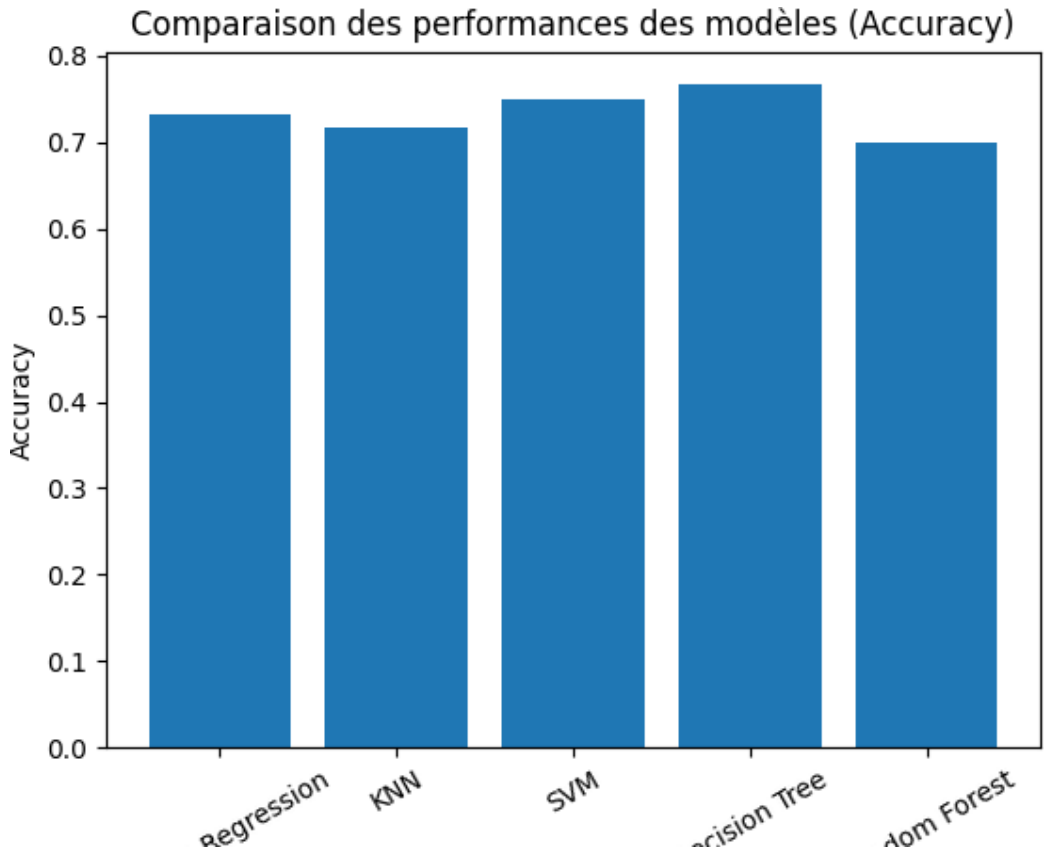


Figure 5.1. Comparaison des performances des modèles de classification.

Ce graphique illustre les scores obtenus par chaque algorithme. On observe que l'algorithme Decision Tree se distingue avec la précision la plus élevée (environ 76%), suivi par le SVM.

En raison de sa structure hiérarchique qui imite le raisonnement médical et de sa performance supérieure sur notre dataset, le Decision Tree a été retenu comme modèle principal pour notre API Flask.

Analyse de la Matrice de Confusion (Modèle retenu)

Pour valider la fiabilité clinique du modèle sélectionné (Decision Tree), nous avons analysé sa matrice de confusion.

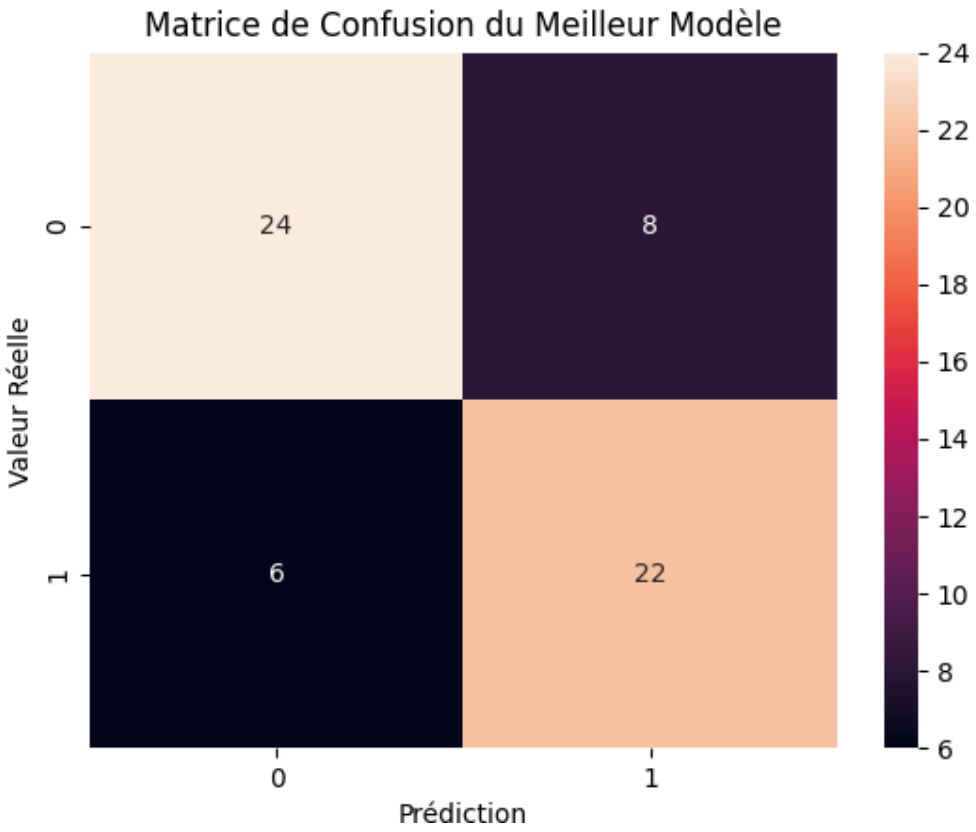


Figure 5.2. Matrice de confusion du modèle Decision Tree.

La matrice permet de visualiser les prédictions correctes et les erreurs. Elle affiche 24 Vrais Négatifs (0) et 22 Vrais Positifs (1).

Le modèle montre une grande efficacité à identifier les cas critiques. Le nombre limité de faux négatifs (6) est un indicateur de sécurité important, minimisant le risque de non-détection d'une pathologie réelle.

Importance des variables (Feature Importance)

Enfin, nous avons étudié quels paramètres physiologiques influencent le plus la décision du modèle pour garantir une "IA explicable".

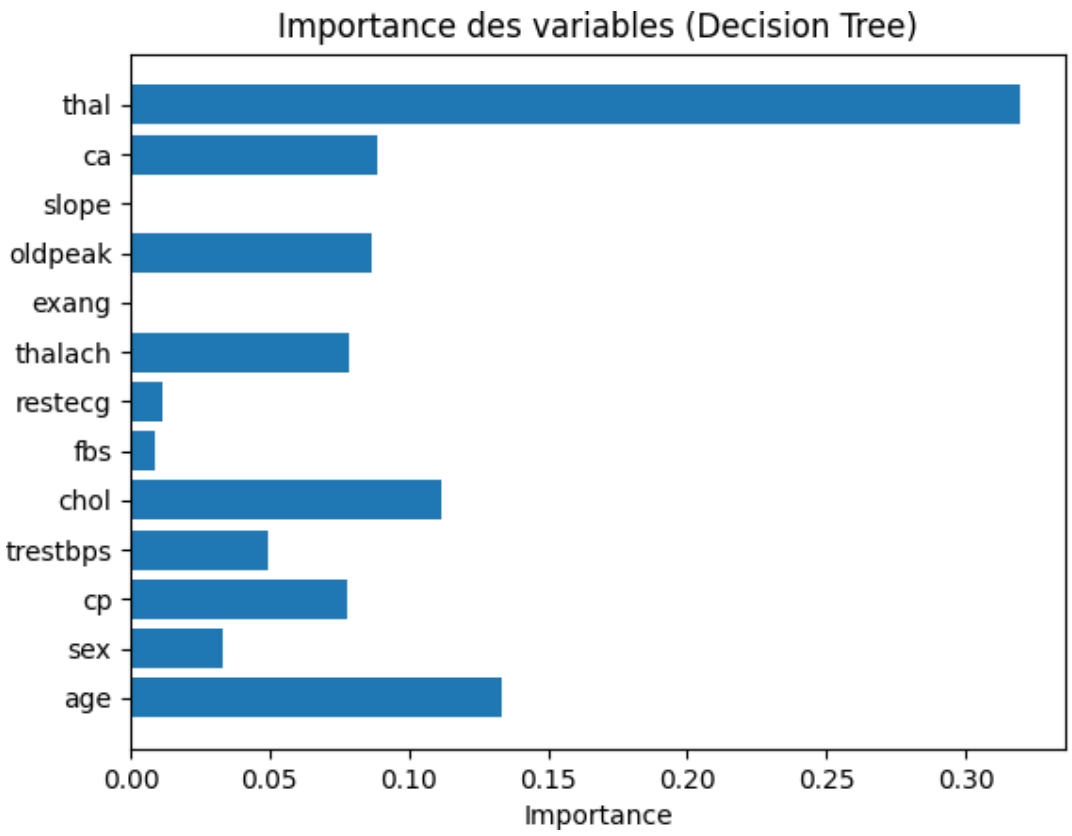


Figure 5.3. Importance des variables cliniques dans le processus de décision.

Ce graphique classe les variables selon leur poids dans la prédiction. On remarque que les variables 'thal' (résultat de la scintigraphie), 'age' et 'chol' (cholestérol) sont les facteurs les plus déterminants.

Ces résultats sont cohérents avec les connaissances médicales actuelles, ce qui renforce la crédibilité du modèle auprès des praticiens utilisant la plateforme.

5.3 Tests du système global

Cette section présente la validation de l'intégration de bout en bout de la plateforme. L'objectif est de démontrer que le flux de données entre le Frontend, le serveur d'IA (Flask), le stockage décentralisé (IPFS) et la Blockchain s'effectue de manière fluide et sécurisée.

5.3.1 Communication entre les modules

Le succès du système dépend de l'interopérabilité des différentes technologies utilisées. Nous avons testé les interfaces de communication suivantes :

— Frontend avec IA (Flask) : Validation de l'envoi des données cliniques en format JSON et de la réception instantanée de la prédiction.

— Frontend avec IPFS : Vérification de la génération correcte du fichier de dossier médical et de son téléversement, confirmé par la réception d'un CID unique.
— Frontend avec Blockchain (MetaMask) : Test de la persistance des données. Une fois le CID reçu, le Frontend déclenche une transaction via Web3.js pour ancrer cette référence sur le Smart Contract.

5.3.2 Scénarios réels

Pour illustrer le fonctionnement opérationnel, nous présentons ci-dessous les captures d'écran des différentes étapes clés du parcours utilisateur.

Phase d'Administration (Gouvernance)

Pour valider le mécanisme de gouvernance, nous avons testé le processus d'autorisation d'un médecin par l'administrateur. Ce scénario confirme que seul le propriétaire du contrat (Owner) peut modifier la liste des praticiens autorisés.

Déclenchement et Signature de la Transaction

La première étape consiste à soumettre l'adresse Wallet du médecin via l'interface d'administration. Comme illustré dans la Figure 5.4, une demande de transaction est automatiquement générée vers MetaMask. On observe l'interaction directe entre le

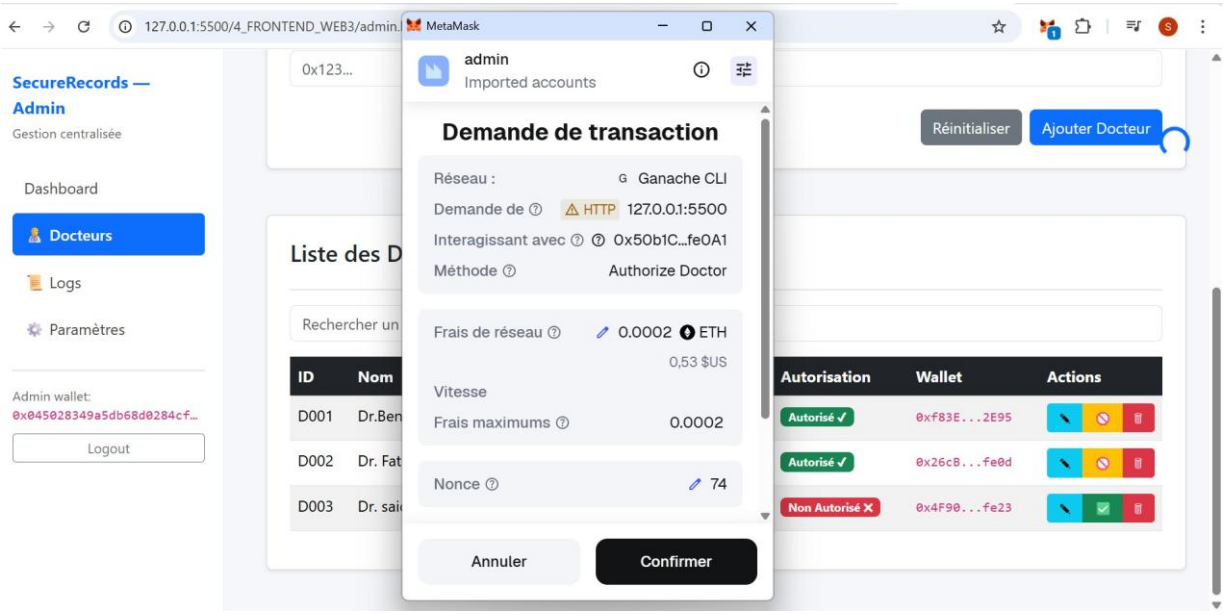


Figure 5.4. Interface d'administration et autorisation d'un professionnel de santé.

Frontend et le fournisseur Web3. L'administrateur doit valider les frais de gaz pour exécuter la fonction authorizeDoctor sur la blockchain Ganache.

Cette étape garantit la sécurité du système : aucune modification n'est possible sans une signature cryptographique valide de l'administrateur.

Confirmation et Mise à jour de l'état

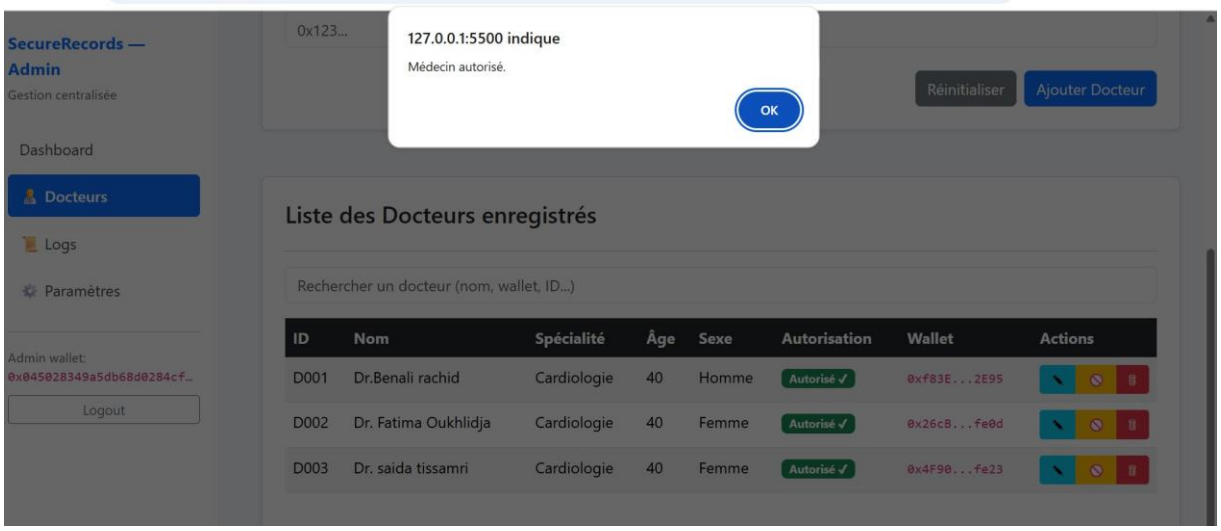


Figure 5.5. Notification de succès et mise à jour de la liste des médecins.

La Figure 5.5 montre le retour de la transaction avec une notification "Médecin autorisé". On remarque également que le statut du médecin dans le tableau de bord passe à "Autorisé" (en vert).
Ce test confirme la réussite de la mise à jour du mapping authorizedDoctors dans le Smart Contract. Désormais, ce médecin possède les droits nécessaires pour enregistrer des diagnostics.
Le succès de ce scénario confirme l'interopérabilité directe entre l'interface Frontend et la Blockchain, garantissant que chaque action administrative est sécurisée et enregistrée en temps réel via Web3.js.

Phase de Diagnostic (Médecin)

Ce scénario illustre le cœur fonctionnel de notre application : l'analyse prédictive d'un risque cardiaque et la sécurisation des résultats. Ce processus se déroule en trois étapes clés validées par nos tests :

Saisie des paramètres cliniques

L'interface médecin (Figure 5.6) permet la saisie structurée des 13 variables physiologiques nécessaires au modèle.

Confirmation et Mise à jour de l'état

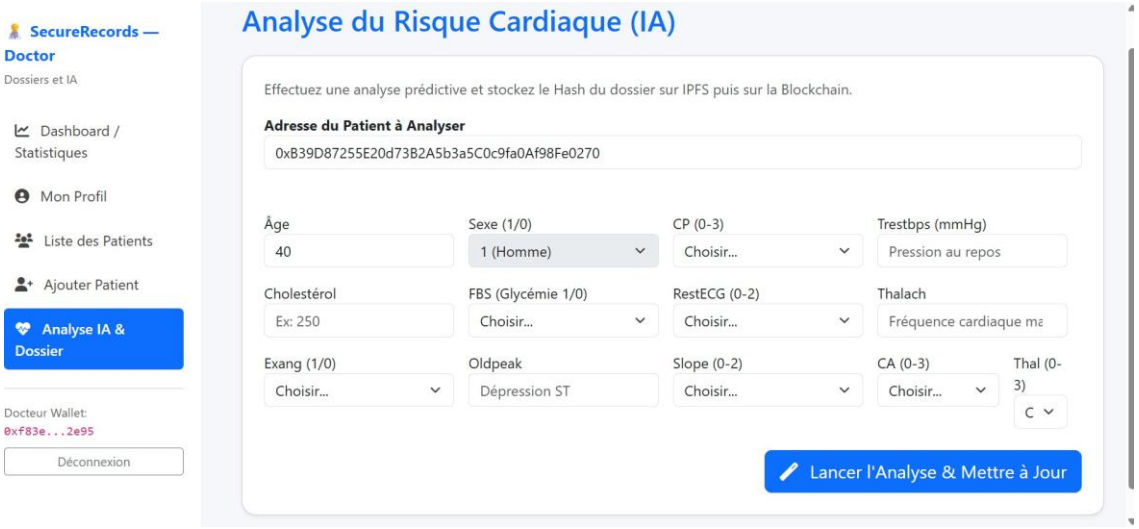


Figure 5.6. Interface de saisie des données cliniques pour l'analyse prédictive.

On y voit l'adresse Wallet du patient ciblée pour l'analyse, assurant que le diagnostic sera lié de manière unique à son identité numérique.

Inférence de l'IA et Alerte de Risque

Une fois le formulaire soumis, les données sont transmises au serveur Flask. Comme le montre la Figure 5.7, le système retourne instantanément une alerte en cas de détection d'un risque élevé.

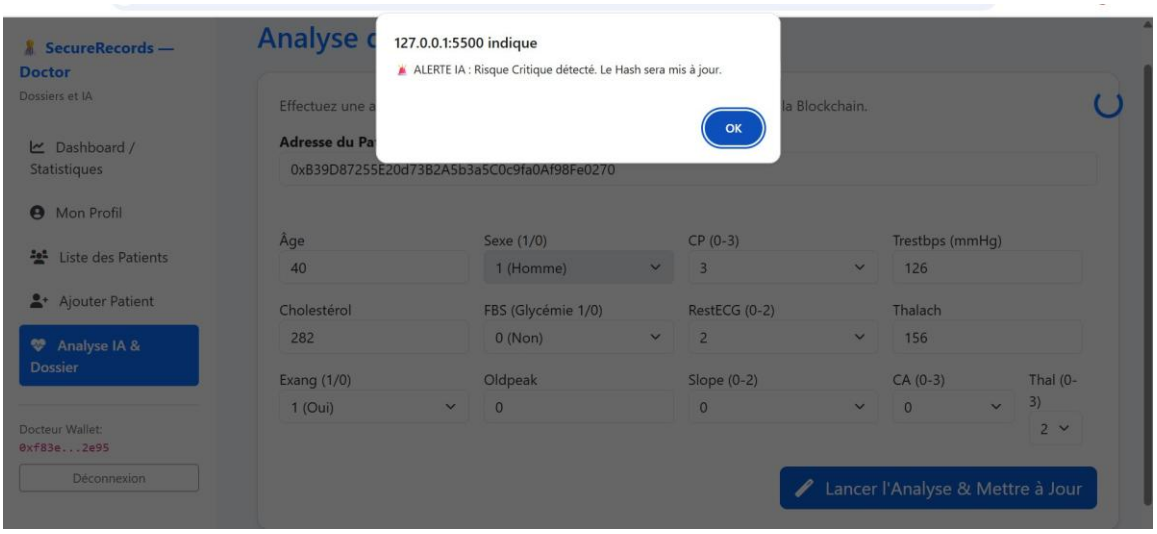


Figure 5.7. Notification de détection de risque critique par le modèle IA.

L'alerte "Risque Critique détecté" confirme que la communication avec l'API IA fonctionne correctement et que le modèle Decision Tree a traité les données avec succès.

Génération du Dossier et Stockage sur IPFS

Après la validation du diagnostic, le système génère un fichier JSON contenant le dossier complet et le téléverse vers le réseau IPFS. La Figure 5.8 affiche le succès de

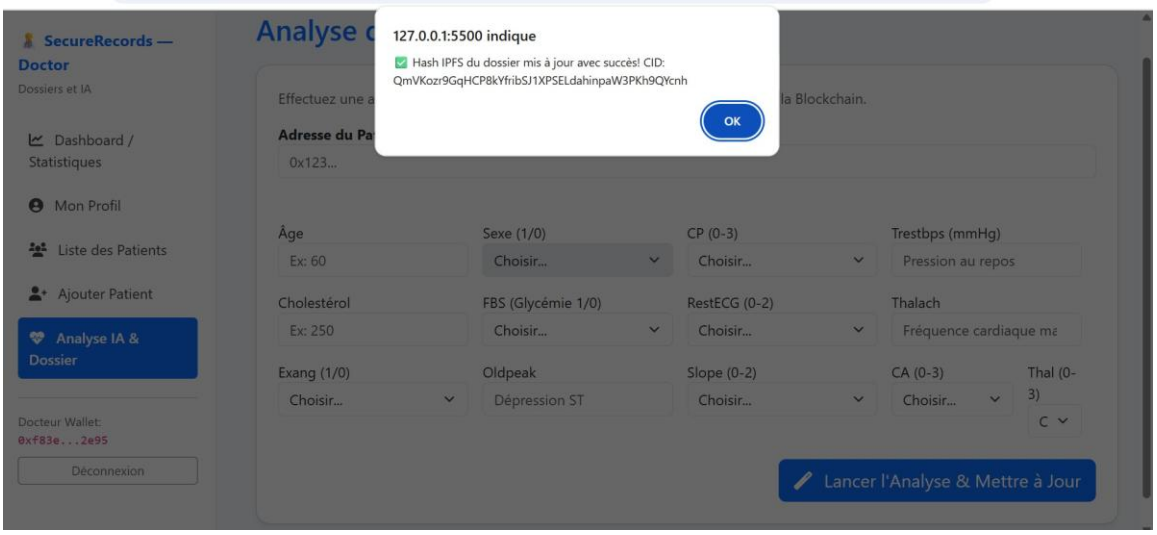


Figure 5.8. Confirmation du téléversement sur IPFS et génération du CID.

l'opération avec la réception d'un CID (Content Identifier) unique. Ce CID sert de preuve d'existence et d'intégrité du dossier médical. Le bon déroulement de ces étapes démontre le succès de l'interopérabilité entre le Frontend, le serveur Flask (IA) et le réseau IPFS. Ce flux garantit que les données cliniques sont non seulement analysées intelligemment en temps réel, mais également décentralisées et prêtes à être ancrées de manière immuable sur la Blockchain.

Phase de Consultation (Patient)

Phase de Consultation et Souveraineté des Données (Patient)

Cette phase finale valide l'accessibilité et la clarté des données pour l'utilisateur final. Le patient, identifié de manière unique par son adresse Ethereum, accède à ses informations médicales en toute transparence.

Authentification et Tableau de Bord

Dès la connexion via MetaMask, le patient accède à un tableau de bord épuré (Figure 5.9).

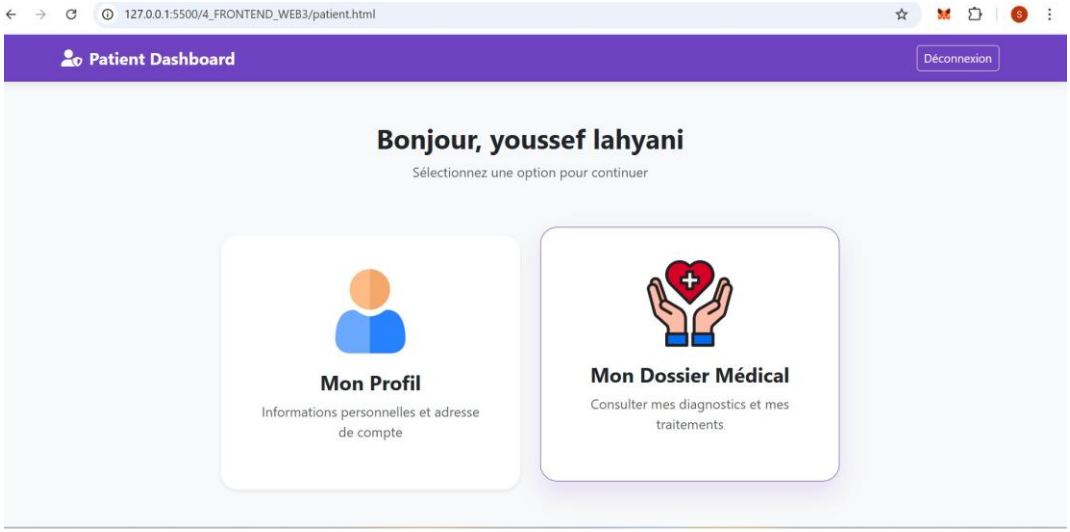


Figure 5.9. Tableau de bord principal de l'interface Patient.

L'interface affiche un message de bienvenue personnalisé ("Bonjour, youssef lahyani"), confirmant que le système a correctement identifié l'utilisateur à partir de son Wallet.

Le dashboard propose deux entrées majeures : "Mon Profil" pour les données identitaires et "Mon Dossier Médical" pour les informations cliniques.

Consultation du Profil Personnel

En accédant à la section "Mon Profil" (Figure 5.10), le patient peut vérifier ses informations de base enregistrées lors de son inscription. On y retrouve le nom complet,

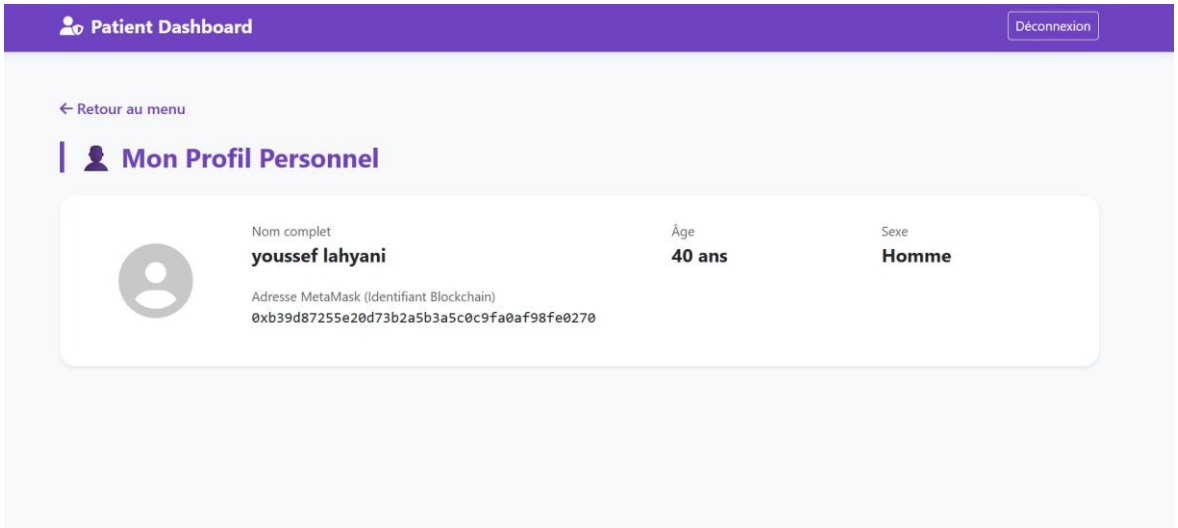


Figure 5.10. Vue détaillée du profil personnel du patient.

l'âge, le sexe, ainsi que l'adresse MetaMask (Identifiant Blockchain).

Cela prouve que les métadonnées stockées sur IPFS sont parfaitement synchronisées avec l'identité numérique du patient.

Visualisation du Rapport Médical et des Traitements

La partie la plus critique est la récupération du rapport médical (Figure 5.11). En cliquant sur "Mon Dossier Médical", le Frontend récupère le CID depuis la Blockchain et télécharge le contenu depuis IPFS. Le patient peut lire son Diagnostic Médical (ex :

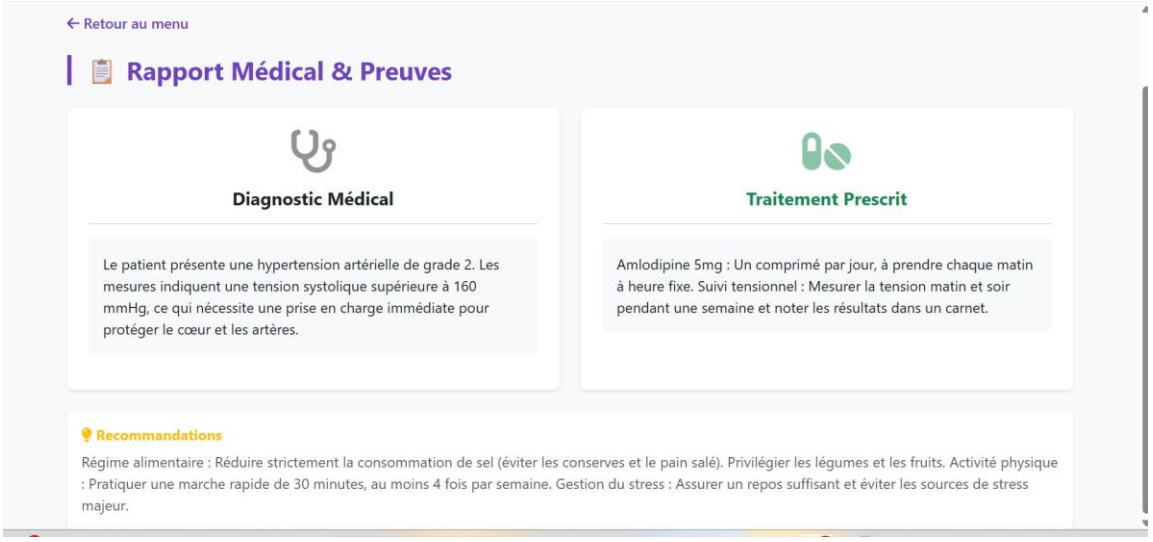


Figure 5.11. Rapport médical détaillé récupéré depuis IPFS

Hypertension artérielle de grade 2) ainsi que le Traitement Prescrit (ex : Amlodipine 5mg) et des recommandations d'hygiène de vie.

Ce test confirme la réussite de la communication Blockchain ⟷ IPFS ⟷ Frontend. Les données complexes saisies par le médecin sont restituées de manière lisible et structurée.

L'affichage correct de ces trois interfaces confirme que le système est capable de reconstruire un dossier médical complet à partir de sources décentralisées. Ce succès valide non seulement l'aspect technique de l'interopérabilité, mais aussi l'objectif social du projet : redonner au patient le plein contrôle et un accès direct à son historique de santé.

5.4 Analyse et discussion des résultats

Après la phase de tests et de validation fonctionnelle de la plateforme, cette section vise à analyser les résultats obtenus et à en discuter la portée. L'objectif est d'évaluer la pertinence des choix technologiques et fonctionnels adoptés, ainsi que d'identifier les apports et les limites du système dans un contexte de gestion sécurisée des dossiers médicaux.

Apports du système

Les résultats obtenus mettent en évidence la valeur ajoutée de l'architecture hybride proposée, reposant sur la combinaison de la Blockchain, de l'intelligence artificielle et du stockage décentralisé via IPFS. Cette approche permet tout d'abord de garantir la souveraineté des

données médicales, en confiant au patient le contrôle exclusif de l'accès à son dossier grâce à l'utilisation de clés privées et de portefeuilles numériques.

L'intégration du modèle d'intelligence artificielle constitue également un apport majeur, en offrant une aide au diagnostic capable de détecter automatiquement les cas critiques. Cette assistance permet de soutenir le médecin dans sa prise de décision et de réduire les risques d'erreurs humaines, notamment dans des situations de forte charge de travail.

Par ailleurs, l'utilisation conjointe d'IPFS et de la Blockchain assure l'intégrité et l'immutabilité des dossiers médicaux. Les données stockées ne peuvent être ni modifiées ni supprimées de manière frauduleuse, garantissant ainsi une traçabilité complète et fiable des informations cliniques. Enfin, le choix d'un stockage hybride, où seuls les identifiants de contenu (CID) sont enregistrés sur la Blockchain, permet de réduire les coûts liés aux frais de transaction tout en maintenant un haut niveau de décentralisation.

Comparaison avec les systèmes classiques

Le tableau suivant résume les différences fondamentales entre notre approche et les systèmes de gestion de dossiers médicaux (DMP) traditionnels :

Table 5.1. Comparaison entre les systèmes classiques et notre solution proposée

Caractéristique	**Systèmes Classiques (Centralisés)**	**Notre Solution (Blockchain + IA)**
Contrôle des données	Administrateur de l'hôpital	Patient (via Clé Privée / Wallet)
Sécurité	Pare-feu (vulnérable aux intrusions)	Cryptographie asymétrique et Immuabilité
Diagnostic	Manuel par le médecin uniquement	Assisté par IA avec détection d'urgence
Disponibilité	Dépendante du serveur central	Haute (Réseau IPFS et Blockchain)
Transparence	Audit interne difficile	Registre public et auditable (Logs Blockchain)

Limites du projet

Malgré les résultats satisfaisants obtenus, certaines limites techniques et fonctionnelles ont été identifiées. Le système dépend fortement de la disponibilité d'une connexion Internet stable pour interagir avec les réseaux Blockchain et IPFS, ce qui peut représenter une contrainte dans des environnements à faible connectivité.

De plus, l'expérience utilisateur demeure perfectible, en particulier pour les patients non familiers avec les technologies Blockchain. L'utilisation de MetaMask et la gestion des frais de transaction peuvent constituer un frein à l'adoption pour certains profils d'utilisateurs, notamment les personnes âgées.

En outre, bien que le modèle d'intelligence artificielle atteigne une précision d'environ 76 %, ses performances pourraient être améliorées par l'exploitation d'un jeu de données plus large et plus diversifié. Cela permettrait d'augmenter la robustesse du diagnostic, notamment pour des cas cliniques rares. Enfin, sur un réseau public, le temps de validation des transactions peut entraîner des délais supplémentaires par rapport à des solutions centralisées traditionnelles.

5.5 Perspectives futures

Ce travail constitue une base solide pour le développement d'un écosystème de santé décentralisé. Plusieurs pistes d'amélioration sont envisagées pour les travaux futurs.

Optimisation du stockage (IPFS)

Nœuds privés : Actuellement, le système utilise des passerelles IPFS publiques. Une perspective intéressante serait de mettre en place un réseau de nœuds IPFS privés au sein des hôpitaux pour garantir une disponibilité maximale et une vitesse de téléchargement accrue des dossiers médicaux.

Chiffrement automatique : Nous envisageons d'intégrer un chiffrement côté client (Client-side encryption) avant l'envoi vers IPFS, afin que même en cas d'accès au fichier sur le réseau décentralisé, seul le patient et son médecin puissent en lire le contenu.

Intelligence Artificielle plus avancée

Apprentissage continu : Une perspective majeure consiste à faire évoluer le modèle vers une architecture capable d'apprendre de manière continue à mesure que de nouveaux cas sont validés par les médecins, augmentant ainsi sa précision au fil du temps.

Diagnostic Multimodal : Nous prévoyons d'étendre les capacités de l'IA pour qu'elle puisse analyser non seulement des données numériques (pouls, cholestérol), mais aussi des images médicales (ECG, IRM) stockées sur IPFS.

Déploiement réel et interopérabilité

Réseau de test public (Testnet) : La prochaine étape serait de déployer les Smart Contracts sur un réseau de test public (comme Sepolia ou Polygon Testnet) pour tester le système dans des conditions de latence réseau réelles.

Application Mobile : Pour améliorer l'expérience utilisateur, le développement d'une application mobile intégrant un Wallet léger permettrait aux patients de consulter leur dossier médical et de recevoir des alertes IA directement sur leurs smartphone

5.6 Conclusion

Les tests réalisés ont permis de valider le bon fonctionnement de la plateforme et l'interopérabilité entre ses différents modules. Les résultats obtenus démontrent la pertinence de l'architecture proposée, combinant Blockchain, IPFS et intelligence artificielle pour assurer la sécurité, la traçabilité et l'aide au diagnostic médical. Malgré certaines limites techniques, la solution constitue une base solide pour le développement futur d'un système de santé décentralisé et intelligent.

Chapitre 6

Conclusion Générale

Ce travail de recherche a été consacré à la conception et à la mise en œuvre d'une plateforme de santé décentralisée, visant à transcender les limites des systèmes hospitaliers classiques par l'intégration synergique de la technologie Blockchain et de l'Intelligence Artificielle. En explorant les défis majeurs liés à la centralisation des données médicales, notamment les vulnérabilités face au piratage et la dépossession des patients de leurs propres informations, nous avons développé une architecture hybride innovante. Cette solution s'appuie sur la Blockchain Ethereum et le langage Solidity pour garantir l'immuabilité et la traçabilité des accès, tandis que le protocole IPFS assure un stockage décentralisé et sécurisé des dossiers volumineux. Parallèlement, l'implémentation d'un modèle de Machine Learning basé sur l'algorithme Decision Tree vient enrichir le système en offrant une assistance au diagnostic en temps réel.

Les résultats obtenus confirment l'atteinte des objectifs fixés au début de cette étude, notamment à travers le déploiement de Smart Contracts rigoureux qui assurent une sécurisation totale des autorisations entre médecins et patients. L'interopérabilité du système a également été validée par une communication fluide entre les interfaces Frontend développées en Web3.js, le serveur d'IA sous Flask et le réseau IPFS. Au-delà de ces prouesses techniques, l'impact de cette solution réside dans sa capacité à redonner une véritable souveraineté numérique au patient, favorisant ainsi une relation de confiance renouvelée avec le corps médical. En proposant une alternative crédible et transparente aux bases de données fragiles, ce projet ouvre la voie à une médecine plus prédictive et accessible. Bien que ce travail constitue une preuve de concept, il démontre avec conviction que la convergence entre la décentralisation et l'intelligence artificielle est une clé essentielle pour la transformation numérique durable du secteur de la santé.

Annexe A

Le Smart Contract (Solidity)

A.1 Structures de données et stockage

Le contrat définit deux structures principales :

Structure Doctor Elle regroupe les informations d'identification du médecin (id, nom, spécialité, âge, sexe) et son adresse wallet utilisée comme identifiant blockchain.

```
// Structure pour stocker les données des médecins
struct Doctor {
    string id;
    string name;
    string specialty;
    uint256 age;
    string sex;
    address wallet;
}
```

Figure A.1. Structure Doctor (extrait du Smart Contract).

Structure Patient C'est la structure centrale du système. Elle contient :

- les informations de base (nom, âge, sexe, wallet),
- le dernier CID IPFS (currentDataHash),
- l'horodatage de mise à jour (lastUpdated),
- l'historique complet des CIDs (diagnosticsHistoryHashes) permettant de suivre l'évolution du dossier.

```
struct Patient {
    string name;
    uint256 age;
    string sex;
    address wallet;
    string currentDataHash; // Hash du dernier fichier médical (contient le diagnostic et le traitement)
    uint256 lastUpdated;
    string[] diagnosticsHistoryHashes; // Historique des Hashes (pour les fichiers de diagnostic précédents)
}
```

Figure A.2. Structure Patient (extrait du Smart Contract).

Indexation et performance

Pour éviter de parcourir des listes complètes, le contrat combine :

— des tableaux dynamiques : doctorsList, patientsList,
— des mappings d'index : doctorIndex, patientIndex (wallet → index),
— un mapping d'autorisations : authorizedDoctors (wallet → bool),
— une relation médecin–patients : doctorPatients (doctor → liste d'adresses patients).

```
// ---------- Storage (Stockage) ----------
Doctor[] private doctorsList;
Patient[] private patientsList;
// Mappings pour la recherche rapide et la vérification d'existence (index+1, 0 = n'existe pas)
mapping(address => uint256) private doctorIndex;
mapping(address => uint256) private patientIndex;
// Mapping pour vérifier l'autorisation des médecins (permission d'écriture des données)
mapping(address => bool) public authorizedDoctors;
mapping(address => bool) public wasAuthorized;
// Relation Médecin-Patient
mapping(address => address[]) public doctorPatients;
```

Figure A.3. Mécanismes de stockage et d'accès (arrays + mappings).

A.2 Contrôle d'accès : Modifiers et logique de sécurité

Le contrat applique un contrôle d'accès explicite :

— onlyAdmin : protège les opérations d'administration (ajout médecin, autorisation, révocation, suppression, transfert de propriété),
— onlyAuthorizedDoctor : limite les opérations d'écriture (ajout patient, mise à jour du hash).

Ainsi, même si un utilisateur est connecté via MetaMask, il ne peut modifier l'état du registre que si son adresse respecte les conditions du contrat.

A.3 Fonctions principales (liées aux besoins fonctionnels)

Administration et gestion des médecins

addDoctor Ajoute un médecin à la liste globale. Le contrat vérifie l'existence via l'indexation afin d'éviter les doublons.

```
function addDoctor(
    string calldata _id,
    string calldata _name,
    string calldata _specialty,
    uint256 _age,
    string calldata _sex,
    address _wallet
  ) external onlyAdmin {
    require(_wallet != address(0), "Adresse de portefeuille invalide");
    require(doctorIndex[_wallet] == 0, "Le medecin existe deja");
    Doctor memory d = Doctor({
        id: _id,
        name: _name,
        specialty: _specialty,
        age: _age,
        sex: _sex,
        wallet: _wallet
    });
    doctorsList.push(d);
    doctorIndex[_wallet] = doctorsList.length; // Stockage de l'index+1

    emit DoctorAdded(_wallet, _id, _name);
}
```

Figure A.4. Fonction addDoctor() (extrait du Smart Contract).

authorizeDoctor Active le droit d'écriture pour un médecin (mise à jour de authorizedDoctors). Sans cette étape, un médecin ne peut pas ajouter de patient ni mettre à jour le CID.

```
function authorizeDoctor(address _wallet) external onlyAdmin {
    require(doctorIndex[_wallet] != 0, "Medecin non enregistre");
    authorizedDoctors[_wallet] = true;
    wasAuthorized[_wallet] = true;
    emit DoctorAuthorized(_wallet);
}
```

Figure A.5. Fonction authorizeDoctor() (extrait du Smart Contract).

revokeDoctor Retire l'autorisation d'écriture (utile en cas de départ, suspension, ou incident de sécurité).

```
function revokeDoctor(address _wallet) external onlyAdmin {
    require(doctorIndex[_wallet] != 0, "Medecin non enregistre");
    authorizedDoctors[_wallet] = false;
}
```

Figure A.6. Fonction revokeDoctor() (extrait du Smart Contract).

Gestion des patients et mise à jour du CID

addPatient Création d'un dossier patient par un médecin autorisé, et association patient → médecin dans doctorPatients.

```
function addPatient(
    string calldata _name↑,
    uint256 _age↑,
    string calldata _sex↑,
    address _wallet↑
  ) external onlyAuthorizedDoctor {
    require(_wallet↑ != address(0), "Adresse de portefeuille invalide");
    require(patientIndex[_wallet↑] == 0, "Le patient existe deja");
    Patient memory newP = Patient({
        name: _name↑,
        age: _age↑,
        sex: _sex↑,
        wallet: _wallet↑,
        currentDataHash: "", // Hash vide au depart
        lastUpdated: block.timestamp,
        diagnosticsHistoryHashes: new string[](0)
    });
    patientsList.push(newP);
    patientIndex[_wallet↑] = patientsList.length; // index+1
    doctorPatients[msg.sender].push(_wallet↑);
    emit PatientAdded(_wallet↑, _name↑);
}
```

Figure A.7. Fonction addPatient() (extrait du Smart Contract).

updatePatientDataHash Fonction centrale : elle reçoit le CID IPFS (rapport JSON), met à jour currentDataHash, met à jour lastUpdated, et ajoute le CID dans l'historique diagnosticsHistoryHashes.

```
function updatePatientDataHash(
    address _wallet↑,
    string calldata _newDataHash↑
  ) external onlyAuthorizedDoctor {
    require(patientIndex[_wallet↑] != 0, "Patient non trouve");
    require(bytes(_newDataHash↑).length > 0, "Le Hash des donnees ne peut pas etre vide");
    Patient storage p = patientsList[patientIndex[_wallet↑] - 1];
    // Mise a jour des champs
    p.currentDataHash = _newDataHash↑;
    p.lastUpdated = block.timestamp;
    p.diagnosticsHistoryHashes.push(_newDataHash↑); // Sauvegarde du Hash dans l'historique

    emit PatientDataHashUpdated(_wallet↑, _newDataHash↑, block.timestamp);
}
```

Figure A.8. Fonction updatePatientDataHash() (extrait du Smart Contract).

Consultation et transparence

getPatientRecord Permet de lire un dossier : le contrat impose que l'appelant soit **le patient lui-même** ou **un médecin autorisé**. La fonction retourne les métadonnées et les CIDs (courant + historique).

```
function getPatientRecord(address _wallet) external view returns (
    string memory name,
    uint256 age,
    string memory sex,
    string memory currentDataHash,
    uint256 lastUpdated,
    string[] memory diagnosticsHistoryHashes
  ) {
    require(patientIndex[_wallet] != 0, "Patient non trouve");
    // Verification de l'autorisation d'acces: doit etre le patient lui-même ou un medecin autorise
    require(msg.sender == _wallet || authorizedDoctors[msg.sender], "Acces refuse a ce dossier");
    Patient storage p = patientsList[patientIndex[_wallet] - 1];
    return (
        p.name,
        p.age,
        p.sex,
        p.currentDataHash, // On retourne uniquement le Hash
        p.lastUpdated,
        p.diagnosticsHistoryHashes
    );
}
```

Figure A.9. Fonction getPatientRecord() (extrait du Smart Contract).

getMyPatientsAddresses Permet au médecin de récupérer uniquement la liste des patients qu'il a ajoutés, utilisée côté interface médecin pour afficher le tableau de suivi.

```
function getMyPatientsAddresses() public view returns (address[] memory) {
    return doctorPatients[msg.sender];
}
```

Figure A.10. Fonction getMyPatientsAddresses() (extrait du Smart Contract).

Annexe B

Implémentation de l'Intelligence Artificielle

Ce script Python constitue le pont entre l'interface utilisateur et le modèle prédictif. Il expose une API REST qui reçoit les paramètres cliniques, les normalise et retourne une prédiction de risque.

B.1 Entraînement du modèle

```python
models = {
    'Logistic Regression': LogisticRegression(solver='liblinear', random_state=42),
    'K-Nearest Neighbors (KNN)': KNeighborsClassifier(n_neighbors=5),
    'Support Vector Machine (SVC)': SVC(kernel='linear', random_state=42),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42)
}
results = {}
print("--- Début de l'entraînement et de l'évaluation des modèles ---")
# --- 4. Entraînement et Évaluation ---
best_model_name = ""
best_accuracy = 0
best_model = None
for name, model in models.items():
    # Entraînement
    model.fit(X_train, y_train)
    # Prédiction sur l'ensemble de test
    y_pred = model.predict(X_test)
    # Évaluation (Précision/Accuracy)
    accuracy = accuracy_score(y_test, y_pred)
    results[name] = accuracy
    print(f"✔ {name}: Précision = {accuracy:.4f}")
    # Déterminer le meilleur modèle
    if accuracy > best_accuracy:
        best_accuracy = accuracy
        best_model_name = name
        best_model = model
```

Figure B.1. Comparaison multi-algorithmes et sélection du modèle de classification optimal.

Cette capture d'écran illustre la phase de Benchmarking et d'entraînement des modèles de Machine Learning. Le script met en évidence l'utilisation d'une approche multi-algorithmes, incluant la Régression Logistique, les K-Plus Proches Voisins (KNN) et les Forêts Aléatoires. Cette étape permet de comparer systématiquement la précision de chaque classifieur sur l'ensemble de test afin de sélectionner dynamiquement le modèle le plus performant pour le diagnostic cardiaque. Une telle méthodologie garantit

la robustesse du système en s'appuyant sur des mesures de performance statistiques objectives avant la phase de déploiement.

B.2 API de prédiction

```
FEATURES = ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', '
@app.route('/predict_risk', methods=['POST'])
def predict():
    if model is None or scaler is None:
        return jsonify({'error': 'Le serveur IA n\'a pas pu charger le modèle.'}), 500
    try:
        data = request.json
        if not all(feature in data for feature in FEATURES):
            return jsonify({'error': 'Données d\'entrée manquantes ou incomplètes.'}), 400
        input_data = [data[feature] for feature in FEATURES]
        input_array = np.array([input_data], dtype=float)
        input_scaled = scaler.transform(input_array)
        prediction = model.predict(input_scaled)
        result = int(prediction[0])
        return jsonify({'risk_prediction': result})
    except Exception as e:
        print(f"Erreur interne lors de la prédiction: {e}")
        return jsonify({'error': f'Erreur de traitement des données: {e}'}), 500
```

Figure B.2. Fonction getMyPatientsAddresses() (extrait du Smart Contract).

Cette capture d'écran illustre l'implémentation de l'API d'inférence développée avec le framework Flask, constituant le pivot technologique entre le modèle d'intelligence artificielle et l'interface utilisateur. Le code met en évidence un flux de traitement rigoureux commençant par la définition des caractéristiques cliniques nécessaires, suivies de la récupération des données en temps réel via des requêtes JSON. Une étape déterminante de normalisation est effectuée grâce à l'outil **scaler.transform** afin d'ajuster les données brutes avant de solliciter la fonction **model.predict**. Enfin, le script assure la transmission sécurisée du résultat de la prédiction du risque cardiaque vers le Frontend, démontrant ainsi une intégration fluide et opérationnelle du modèle prédictif au sein de l'architecture globale du projet.

B.3 Évaluation et Visualisation

```
y_pred = model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Matrice de Confusion du Meilleur Modèle")
plt.xlabel("Prédiction")
plt.ylabel("Valeur Réelle")
plt.show()
```

Figure B.3. Génération de la matrice de confusion pour l'évaluation des performances diagnostiques.

Cette capture d'écran illustre la phase d'évaluation finale du modèle à travers la génération d'une matrice de confusion à l'aide des bibliothèques Seaborn et Matplotlib. Le script permet de comparer les prédictions du modèle avec les valeurs réelles de

l'ensemble de test, offrant ainsi une vision détaillée de la précision du diagnostic. Cette visualisation est essentielle pour valider la capacité du système à distinguer correctement entre les patients sains et les patients à risque, garantissant ainsi la fiabilité de l'outil d'aide à la décision avant son intégration dans le flux de travail médical décentralisé.

Annexe C

Structure du fichier JSON médical stocké sur IPFS

Figure C.1. Structure des métadonnées médicales au format JSON (Stockage IPFS).

Cette capture d'écran illustre la structure du fichier JSON stocké de manière décentralisée sur le réseau IPFS. Ce format de métadonnées permet d'organiser les informations cliniques du patient ainsi que le résultat du diagnostic généré par l'IA dans un format léger et interopérable. L'intégrité de ce fichier est garantie par son empreinte numérique unique (Hash IPFS), dont la référence est enregistrée sur la Blockchain Ethereum pour assurer une traçabilité immuable et sécurisée des données médicales.

Annexe D

Visualisation des Transactions sur Ganache

```
PS C:\Users\tahri\Desktop\PFE_Medical_Decentralise> CD 3_SMART_CONTRACT_TRUFFLE
PS C:\Users\tahri\Desktop\PFE_Medical_Decentralise\3_SMART_CONTRACT_TRUFFLE> truffle compile
>>

Compiling your contracts...
===========================
> Compiling .\contracts\MedicalRecords.sol
> Compiling .\contracts\MedicalRecords.sol
> Artifacts written to C:\Users\tahri\Desktop\PFE_Medical_Decentralise\3_SMART_CONTRACT_TRUFFLE\build\contracts
> Compiled successfully using:
   - solc: 0.8.20+commit.a1b79de6.Emscripten.clang
PS C:\Users\tahri\Desktop\PFE_Medical_Decentralise\3_SMART_CONTRACT_TRUFFLE> truffle migrate --reset --network developmen
t

Compiling your contracts...
===========================
> Compiling .\contracts\MedicalRecords.sol
> Artifacts written to C:\Users\tahri\Desktop\PFE_Medical_Decentralise\3_SMART_CONTRACT_TRUFFLE\build\contracts
> Compiled successfully using:
   - solc: 0.8.20+commit.a1b79de6.Emscripten.clang


Starting migrations...
======================
> Network name:    'development'
> Network id:      1766610239687
> Block gas limit: 30000000 (0x1c9c380)
```

Figure D.1. Processus de déploiement du Smart Contract MedicalRecords.sol sur Ganache via Truffle.

Cette capture d'écran du terminal de commande documente la phase de compilation et de migration du Smart Contract MedicalRecords.sol vers l'environnement Ganache. Les journaux d'exécution (Logs) confirment le succès de l'opération en utilisant le compilateur Solidity version 0.8.20. Cette étape fournit des métadonnées techniques cruciales, notamment l'identifiant du réseau (Network ID) et la limite de gaz par bloc (Block Gas Limit). Ces informations attestent que l'infrastructure Blockchain est correctement configurée et prête à assurer le stockage immuable des empreintes numériques (Hashes IPFS) liées aux diagnostics médicaux, garantissant ainsi une traçabilité décentralisée et sécurisée au sein du système.

Bibliographie

[1] S. Nakamoto, Bitcoin : A Peer-to-Peer Electronic Cash System, 2008.

[2] G. Wood, Ethereum : A Secure Decentralised Generalised Transaction Ledger, Ethereum Yellow Paper, 2014.

[3] Azaria et al., MedRec : Using Blockchain for Medical Data Access and Permission Management, IEEE Open & Big Data Conference, 2016.

[4] Kuo, Kim & Ohno-Machado, Blockchain distributed ledger technologies for biomedical and health care applications, Journal of the American Medical Informatics Association, 2017.

[5] Hölbl et al., A systematic review of the use of blockchain in healthcare, Symmetry, 2018.

[6] Benet, J., IPFS – Content Addressed, Versioned, P2P File System, 2014.

[7] Zyskind et al., Decentralizing Privacy : Using Blockchain to Protect Personal Data, IEEE Security & Privacy Workshops, 2015.

[8] Buterin, V., A Next-Generation Smart Contract and Decentralized Application Platform, Ethereum White Paper, 2014.

[9] Christidis & Devetsikiotis, Blockchains and Smart Contracts for the Internet of Things, IEEE Access, 2016.

[10] Esteva et al., A guide to deep learning in healthcare, Nature Medicine, 2019.

[11] Rajkomar et al., Machine Learning in Medicine, New England Journal of Medicine, 2019.

[12] Breiman, L., Random Forests, Machine Learning Journal, 2001.

[13] Pedregosa et al., Scikit-learn : Machine Learning in Python, Journal of Machine Learning Research, 2011.

[14] Grinberg, M., Flask Web Development, O'Reilly Media, 2018.

[15] Web3 Foundation, Web3.js Documentation, https ://web3js.readthedocs.io

[16] MetaMask Documentation, https ://metamask.io

[17] HIPAA, Health Insurance Portability and Accountability Act, 1996.

[18] ISO/IEC 27001, Information Security Management Systems, ISO, 2013.

[19] Zhang et al., Blockchain-Based Secure and Trustworthy EHR Sharing System, IEEE Access, 2018.

[20] Islam et al., A Blockchain-Based Secure Healthcare Scheme, IEEE Access, 2019.